

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana
w specjalności Informatyka w Biznesie

Generator tekstowych danych losowych dla zadanego kontekstu wykorzystujący LLM

inż. Jakub Prokopiuk

numer albumu 319002

promotor

dr hab. inż. Bartosz Sawicki

WARSZAWA 2026

Generator tekstowych danych losowych dla zadanego kontekstu wykorzystujący LLM

Streszczenie

Poniższa praca magisterska poświęcona jest zaprojektowaniu i implementacji zaawansowanego systemu do generowania syntetycznych, relacyjnych zbiorów danych, który łączy tradycyjne metody algorytmiczne z możliwościami dużych modeli językowych.

W ramach pracy opracowano aplikację o architekturze mikroserwisowej, wykorzystującą konteneryzację Docker do integracji warstwy frontendowej opartej na bibliotece React, warstwy backendowej zrealizowanej w frameworku FastAPI oraz systemu kolejkowania zadań asynchronicznych Redis i Celery. Takie podejście pozwoliło na zapewnienie skalowalności i wydajności niezbędnej do generowania dużych wolumenów danych bez blokowania interfejsu użytkownika.

Kluczowym elementem rozwiązania jest silnik generujący, który implementuje hybrydowe podejście do tworzenia danych. Wykorzystuje on bibliotekę Faker do danych strukturalnych oraz modele LLM - obejmujące zarówno rozwiązania chmurowe (OpenAI), jak i modele uruchamiane lokalnie - do generowania treści kreatywnych i kontekstowych. System automatycznie analizuje schemat bazy danych i rozwiązuje zależności kluczy obcych w celu zachowania integralności referencyjnej generowanych rekordów.

Zaimplementowane rozwiązanie wykracza poza standardowe generatory plików płaskich, oferując mechanizm bezpośredniego zrzutu danych do popularnych silników bazodanowych, takich jak PostgreSQL, MySQL, MSSQL oraz Oracle. Dodatkowo system wyposażono w moduł uwierzytelniania, zarządzanie projektami w chmurze oraz podgląd postępu generowania w czasie rzeczywistym. Rezultatem pracy jest kompletne, gotowe do wdrożenia narzędzie, wspierające procesy testowania, prototypowania i analityki danych.

Słowa kluczowe: Generowanie danych syntetycznych, LLM, FastAPI, bazy danych, Docker, Sztuczna Inteligencja, Redis, Celery

Textual Data Generator for a Given Context Using LLM

Abstract

This master's thesis is devoted to the design and implementation of an advanced system for generating synthetic relational datasets, which combines traditional algorithmic methods with the capabilities of Large Language Models.

As part of the thesis, an application with a microservice architecture was developed, utilizing Docker containerization to integrate a React-based frontend, a FastAPI backend, and an asynchronous task queuing system based on Redis and Celery. This approach ensured the scalability and performance necessary to generate large volumes of data without blocking the user interface.

A key element of the solution is the generation engine, which implements a hybrid approach to data creation. It employs standard libraries for structured data and Large Language Models—including both cloud-based solutions and local instances—to generate creative and contextual content. The system automatically analyzes the database schema, resolving foreign key dependencies to maintain the referential integrity of the generated records.

The implemented solution extends beyond standard flat-file generators, offering a mechanism for direct data dumps to popular database engines such as PostgreSQL, MySQL, MSSQL, and Oracle. Additionally, the system is equipped with an authentication module, cloud project management capabilities, and real-time generation progress monitoring via WebSocket protocol. The result of the work is a complete and ready-to-deploy tool that supports testing, prototyping, and data analytics processes.

Keywords: Synthetic Data Generation, LLM, FastAPI, Databases, Docker, Artificial Intelligence, Redis, Celery

Spis treści

1	Wstęp	9
2	Analiza i wybór technologii	13
2.1	Język programowania backend: Python	13
2.2	Framework webowy: FastAPI vs Flask vs Django	13
2.3	Warstwa prezentacji: React	14
2.4	Infrastruktura dodatkowa	15
2.5	Konteneryzacja: Docker	16
3	Analiza wymagań i funkcjonalność systemu	17
3.1	Wymagania funkcjonalne	17
3.2	Wymagania нефункционалне	18
4	Metodyka pracy	21
4.1	Wykorzystanie wsparcia sztucznej inteligencji	21
4.2	Wkład autorski i decyzyjność	22
5	Implementacja	23
5.1	Architektura systemu	23
5.2	Implementacja backendowa	24
5.3	Implementacja frontendowa	25
5.4	Implementacja algorytmów	35
5.5	Infrastruktura i konteneryzacja	36
6	Weryfikacja systemu, wyzwania i wnioski	39
6.1	Testy poprawności i bezpieczeństwa	39
6.1.1	Testy jednostkowe i integracyjne backendu	39
6.1.2	Testy manualne interfejsu użytkownika	40
6.1.3	Testy End-to-End (End-to-End (E2E))	41
6.1.4	Testy bezpieczeństwa	41
6.2	Wyzwania implementacyjne	42
6.2.1	Informowanie użytkownika o postępie	42
6.2.2	Wymuszanie unikalności danych z modelu LLM	42

6.2.3	Zależności cykliczne	43
6.2.4	Odporność na błędy łączności z API	43
6.3	Wydajność i optymalizacja generowania	43
6.3.1	Algorytm dwufazowy i przetwarzanie wsadowe	44
6.3.2	Dobór parametrów optymalizacji	44
6.3.3	Porównanie wydajności Faker i LLM	45
6.3.4	Alternatywny silnik — vLLM	45
6.3.5	Skalowalność dla dużych zbiorów danych	46
6.4	Podsumowanie i wnioski	48
6.4.1	Osiągnięcia	48
6.4.2	Dalszy rozwój aplikacji	48

Bibliografia	51
---------------------	-----------

Wykaz skrótów i symboli	55
--------------------------------	-----------

Spis rysunków	57
----------------------	-----------

Spis tabel	59
-------------------	-----------

Spis załączników	61
-------------------------	-----------

Rozdział 1

Wstęp

We współczesnym świecie dane stanowią jeden z najcenniejszych zasobów przedsiębiorstw, napędzając rozwój systemów informatycznych, analityki biznesowej oraz uczenia maszynowego. Jednocześnie, wraz ze wzrostem wolumenu przetwarzanych informacji, rośnie ryzyko związane z ich bezpieczeństwem i zgodnością z przepisami. W dobie rygorystycznych regulacji o ochronie danych osobowych (GDPR) [3] oraz rosnącej złożoności systemów informatycznych, pozyskanie realistycznych danych testowych — zachowujących spójność logiczną, referencyjną i kontekstową — stanowi istotne wyzwanie współczesnej inżynierii oprogramowania.

Problem ten ujawnia się szczególnie wyraźnie w procesach testowania i prototypowania aplikacji opartych o relacyjne bazy danych. Zespoły deweloperskie potrzebują zbiorów, które odzwierciedlają strukturę docelowego systemu (tabele, klucze obce, ograniczenia), ale jednocześnie zawierają treści wiarygodne semantycznie — np. recenzje produktów powiązane z ich kategorią lub adresy zgodne z krajem użytkownika. Bez takiej spójności trudno jest wiarygodnie testować zaawansowane algorytmy analityczne, moduły raportowe czy systemy rekomendacyjne.

Tradycyjne podejście do testowania oprogramowania często opierało się na wykorzystaniu kopii produkcyjnych baz danych, poddanych procesom anonimizacji lub maskowania. Metody te, choć skuteczne w zachowaniu struktury danych, niosą ze sobą ryzyko reidentyfikacji osób fizycznych [18] oraz są trudne do skalowania w środowiskach Continuous Integration / Continuous Deployment (CI/CD). Z drugiej strony proste generatory danych losowych (ang. *random data generators*) często tworzą zbiory niespójne semantycznie, co ogranicza ich przydatność w scenariuszach wymagających realizmu treści, a nie wyłącznie poprawności składniowej rekordów.

Przełomem w dziedzinie generowania treści stało się upowszechnienie dużych modeli językowych (Large Language Model (LLM)). Modele takie jak GPT-4 czy Llama 3 potrafią generować tekst o wysokim stopniu realizmu, uwzględniając zadany kontekst [17]. Jednak samo użycie LLM do masowego wypełniania tabel jest procesem kosztownym obliczeniowo i wolnym, co czyni je niepraktycznym przy generowaniu milionów rekordów. Dodatkowo modele probabilistyczne mogą naruszać integralność referencyjną lub strukturę formatów wymiany danych (JavaScript Object Notation (JSON), Structured Query Language (SQL)). Z tego powodu uzasadnione jest

poszukiwanie rozwiązań hybrydowych, łączących szybkość klasycznych algorytmów pseudolosowych z kreatywnością i rozumieniem kontekstu przez sztuczną inteligencję.

Aby precyzyjnie określić miejsce proponowanego narzędzia, konieczne jest zestawienie dostępnych na rynku podejść do generowania danych syntetycznych.

Przegląd istniejących rozwiązań

Rynek narzędzi do generowania danych syntetycznych jest obecnie dynamicznie rozwijającym się obszarem, który można podzielić na trzy główne kategorie: rozwiązania algorytmiczne, rozwiązania oparte na uczeniu maszynowym oraz rozwiązania wykorzystujące modele językowe. Poniższa analiza przedstawia wiodące narzędzia w tych kategoriach, wskazując ich zalety oraz ograniczenia w kontekście generowania spójnych relacyjnie baz danych.

Rozwiązania algorytmiczne i quasi-losowe

Jeszcze do niedawna były to najpopularniejsze narzędzia do generowania danych syntetycznych. Sztandarowe narzędzia w tej kategorii to m.in.:

Biblioteka Faker

Faker [4] to biblioteka open-source dostępna w wielu językach programowania (Python, JavaScript, Ruby), umożliwiająca generowanie różnorodnych danych losowych, takich jak imiona, adresy, numery telefonów czy dane finansowe.

- **Zalety:** Bardzo wysoka wydajność (generowanie odbywa się w czasie rzeczywistym), brak kosztów (narzędzie jest open-source), determinizm (możliwość odtworzenia danych przy użyciu seed).
- **Wady:** Brak kontekstualnej spójności między generowanymi danymi. Wygenerowana „recenzja produktu” to zazwyczaj zbiór losowych słów z łaciny lub zdań niemających sensu logicznego. Znaczącą wadą jest również brak wbudowanej obsługi złożonych relacji między tabelami (użytkownik musi sam zaprogramować logikę kluczy obcych).

Mockaroo

Mockaroo to popularne narzędzie webowe (Software as a Service (SaaS)), które pozwala definiować schematy danych w GUI i pobierać je jako Comma-Separated Values (CSV)/SQL/JSON.

- **Zalety:** Intuicyjny interfejs użytkownika, szeroki zakres typów danych, możliwość eksportu do różnych formatów, wsparcie dla relacji między tabelami za pomocą prostych skryptów pisanych w Ruby.
- **Wady:** W wersji darmowej generowanie limitowane jest do 1000 wierszy. Mimo zaawansowania, nadal opiera się głównie na losowaniu ze słowników. Funkcje sztucznej inteligencji są ograniczone lub dodatkowo płatne.

Rozwiązania oparte na modelach generatywnych

Ta grupa narzędzi (np. Gretel.ai [7], Mostly AI [15]) wiąże się z istotnymi wyzwaniami dotyczącymi prywatności danych. Działają one na zasadzie: „Wgraj nam swoją prawdziwą bazę danych, a my nauczymy model jej schematu i wygenerujemy nową, która wygląda tak samo, ale nie zawiera danych osobowych”.

- **Zalety:** Potrafią generować dane o wysokim stopniu realizmu, zachowując statystyczne właściwości oryginalnego zbioru.
- **Wady:** Wymagają dostępu do rzeczywistych danych, co może naruszać przepisy o ochronie danych osobowych. Wymagają również danych wejściowych, przez co nie nadają się do tworzenia danych do nowo powstających systemów, gdzie nie ma jeszcze żadnej bazy danych, na której można by uczyć model. Koszty licencyjne są wysokie, a skalowalność ograniczona przez infrastrukturę dostawcy.

Rozwiązania oparte na modelach językowych

Najnowsza kategoria narzędzi do generowania danych syntetycznych wykorzystuje możliwości LLM, takich jak GPT-4 czy Claude. Przykładowo, można poprosić model o wygenerowanie danych do tabeli „Recenzje produktów”, podając mu kontekst (np. kategorie produktów, cechy użytkowników).

- **Zalety:** Wysoki realizm tekstowy, kreatywność, rozumienie kontekstu.
- **Wady:** Wysokie koszty operacyjne, ograniczenia szybkości, model może „wymyślić” nieistniejące ID lub zgubić strukturę JSON/SQL przy dużej ilości danych. Przez ograniczenia kontekstowe, generowanie dużych zbiorów danych wymaga podziału na mniejsze partie, co komplikuje zachowanie spójności referencyjnej między tabelami.

Analiza powyższych kategorii wskazuje na istotną lukę. Brakuje narzędzia, które nie wymaga danych wejściowych, zapewnia wysoki realizm i kontekstowość danych tekstowych, gwarantuje ścisłą integralność relacyjną przy dużej skali oraz pozostaje efektywne kosztowo — w szczególności nie wykorzystuje drogich wywołań LLM do generowania prostych pól strukturalnych, takich jak data urodzenia czy numer telefonu.

Poniższa tabela (Tabela 1) zestawia omówione podejścia z rozwiązaniem autorskim opracowanym w ramach niniejszej pracy.

Cecha	Faker	Mockaroo	Gretel.ai	Rozw. autorskie
Generowanie strukturalne	TAK	TAK	TAK	TAK
Generowanie kontekstowe AI	NIE	Ograniczone	TAK	TAK
Obsługa relacji (FK)	Manualna	Skryptowa	Automatyczna	Automatyczna
Prywatność (Lokalne modele)	-	NIE	Częściowo	TAK (Ollama)
Wymaga danych wejściowych	NIE	NIE	TAK	NIE
Koszt	Darmowy	Freemium	Płatny	Darmowy

Tabela 1. Porównanie narzędzi do generowania danych

Cel i zakres pracy

Celem niniejszej pracy jest zaprojektowanie i implementacja narzędzia służącego do proceduralnego generowania relacyjnych zbiorów danych testowych. Proponowane rozwiązanie integruje wydajne algorytmy pseudolosowe (biblioteka Faker) do generowania danych strukturalnych oraz modele LLM — zarówno chmurowe, jak i uruchamiane lokalnie — do tworzenia danych kontekstowych. System został zaprojektowany w architekturze mikroserwisowej, wykorzystując konteneryzację Docker, kolejkę zadań Redis oraz asynchroniczne przetwarzanie zadań (Celery), co umożliwia symulację środowisk produkcyjnych o wysokiej złożoności bez blokowania interfejsu użytkownika.

Zakres pracy obejmuje:

- analizę dostępnych technologii i uzasadnienie wyboru stosu implementacyjnego,
- specyfikację wymagań funkcjonalnych i нефunkcjonalnych systemu,
- opis metodyki realizacji pracy,
- implementację aplikacji webowej wraz z silnikiem generowania danych,
- testy poprawności, wydajności oraz bezpieczeństwa,
- omówienie napotkanych problemów implementacyjnych i wniosków końcowych.

Praca została podzielona na następujące rozdziały. Rozdział dotyczący analizy technologii przedstawia porównanie frameworków backendowych i frontendowych. Kolejny rozdział definiuje wymagania i funkcjonalność systemu. Następnie opisano metodykę pracy oraz szczegóły implementacji. Pracę zamyka rozdział obejmujący weryfikację systemu (testy poprawności, bezpieczeństwa i wydajności), omówienie napotkanych wyzwań implementacyjnych oraz podsumowanie uzyskanych rezultatów. Kod źródłowy aplikacji zamieszczono w załączniku.

Rozdział 2

Analiza i wybór technologii

Wybór odpowiednich narzędzi jest kluczowym elementem procesu projektowania każdego systemu informatycznego. Decyzje podjęte na tym etapie mają bezpośredni wpływ na wydajność, skalowalność, bezpieczeństwo oraz łatwość utrzymania aplikacji w przyszłości. W niniejszym rozdziale przedstawiono analizę dostępnych rozwiązań oraz uzasadniono wybór konkretnych technologii wykorzystanych w projekcie.

2.1 Język programowania backend: Python

Język Python został wybrany jako fundament warstwy serwerowej. Decyzja ta podyktowana była kilkoma czynnikami:

1. Ekosystem AI/ML: Python jest niekwestionowanym liderem w dziedzinie sztucznej inteligencji. Biblioteki takie jak `openai` czy `pandas` posiadają natywne wsparcie dla tego języka, co znacznie upraszcza integrację z modelami LLM.
2. Generatory danych: Dostępność biblioteki `Faker`, która jest standardem w generowaniu danych testowych.
3. Szybkość prototypowania: Zwięzła składnia Pythona pozwala na szybkie tworzenie i testowanie nowych funkcjonalności.

2.2 Framework webowy: FastAPI vs Flask vs Django

Do budowy Application Programming Interface (API) rozważano trzy najpopularniejsze frameworki Pythonowe: Django[1], Flask[25] oraz FastAPI[22]. Różnią się one przede wszystkim filozofią projektową. Django to duży framework, oferujący gotowe rozwiązania (ORM, panel administracyjny, uwierzytelnianie), co jednak wiąże się z dużym narzutem. Flask i FastAPI to mikroframeworki, dostarczające jedynie niezbędnych narzędzi do obsługi żądań HTTP, pozostawiając programiście swobodę w doborze bibliotek (np. do bazy danych).

Cecha	Django	Flask	FastAPI
Wydajność	Średnia	Średnia	Wysoka
Obsługa Async	Częściowa	Ograniczona	Natywna
Typowanie	Opcjonalne	Brak	Wymuszone (Pydantic)

Tabela 2. Porównanie frameworków webowych Python

Wybrano FastAPI ze względu na natywną obsługę asynchroniczności (kluczowe przy długotrwałych zapytaniach do LLM) oraz automatyczną walidację danych przy użyciu modeli Pydantic, co znacząco podnosi bezpieczeństwo typów.

2.3 Warstwa prezentacji: React

Dla interfejsu użytkownika zdecydowano się na model oparty na technologii jednej strony (Single Page Application (SPA))[16], co pozwala na płynne przechodzenie między widokami bez konieczności przeładowywania całej strony. W procesie wyboru technologii rozważano trzy wiodące rozwiązania: Angular[6], Vue.js[36] oraz React [13].

Aby dokonać obiektywnego wyboru, zdefiniowano następujące kryteria porównawcze:

1. Próg wejścia: Poziom trudności oraz czas wymagany do efektywnego rozpoczęcia pracy z technologią.
2. Model danych: Sposób przepływu danych (jednokierunkowy vs dwukierunkowy) i jego wpływ na przewidywalność stanu aplikacji.
3. Elastyczność: Poziom swobody w doborze bibliotek towarzyszących (np. do routingu, zarządzania stanem).

Cecha	Angular	Vue.js	React
Próg wejścia	Wysoki	Niski	Średni
Model danych	Dwukierunkowy	Dwukierunkowy	Jednokierunkowy
Elastyczność	Niska	Wysoka	Wysoka

Tabela 3. Porównanie technologii frontendowych

Analiza alternatyw:

- Angular: Jest kompletnym rozwiązaniem typu „wszystko w jednym”. Choć oferuje gotowe narzędzia do większości zadań, narzuca sztywne ramy architektoniczne i wymaga znajomości TypeScript oraz biblioteki RxJS. Dla projektu, w którym priorytetem jest elastyczność i szybkość prototypowania, okazał się zbyt ciężki i skomplikowany.
- Vue.js: Pod wieloma względami (np. wydajności) jest zbliżony do Reacta i posiada niższy próg wejścia. Jednak ekosystem Reacta, w tym dostępność gotowych komponentów integrujących

się z nowoczesnymi narzędziami, jest znacznie bogatszy, co zadecydowało o jego odrzuceniu na korzyść Reacta.

Ostatecznie wybrano React, kierując się następującymi atutami:

- Komponentowość: Ułatwia budowanie skomplikowanych interfejsów z małych, niezależnych i reużywalnych części (np. formularz pola, modal konfiguracji).
- Wirtualny Document Object Model (DOM): Zapewnia wysoką wydajność, minimalizując kosztowne operacje na rzeczywistym drzewie DOM, co jest kluczowe przy wyświetlaniu dużych zestawów wygenerowanych danych.
- Bogaty ekosystem: Ogromna społeczność i dostępność gotowych bibliotek (takich jak Lucide React) znacząco przyspiesza proces budowania interfejsu.

Jako narzędzie budujące i środowisko uruchomieniowe wybrano Vite [35], rezygnując z tradycyjnego rozwiązania jakim jest Webpack. Decyzja ta wynikała z fundamentalnych różnic w sposobie działania obu narzędzi:

- Szybkość startu: Vite wykorzystuje natywne moduły ES[12] w przeglądarce, co pozwala na natychmiastowe uruchomienie serwera deweloperskiego, niezależnie od rozmiaru aplikacji. Webpack musi zbudować cały kod aplikacji przed startem.
- Hot Module Replacement (HMR): W Vite podmiana modułów następuje niemal natychmiastowo, co znacząco przyspiesza tworzenie aplikacji.
- Optymalizacja kodu: Vite w trybie produkcyjnym wykorzystuje narzędzie Rollup[24], co pozwala na zaawansowane optymalizacje, takie jak tree-shaking (usuwanie martwego kodu) oraz lazy loading (dzielenie kodu na mniejsze fragmenty ładowane na żądanie). Dzięki temu finalny plik aplikacji jest mniejszy i łąduje się szybciej.

2.4 Infrastruktura dodatkowa

Mimo że generowane dane mogą mieć różną strukturę, zdecydowano się na relacyjną bazę danych PostgreSQL [30] do przechowywania metadanych systemu (użytkownicy, projekty, historia zadań). Głównym argumentem była potrzeba zachowania ścisłej spójności transakcyjnej (ACID [8]) przy zarządzaniu projektami oraz natywne wsparcie dla typu danych JSONB. Pozwala to na elastyczne przechowywanie konfiguracji schematów (które są strukturami JSON) przy jednoczesnym zachowaniu zalet języka SQL do zapytań analitycznych.

Generowanie tysięcy rekordów, szczególnie z użyciem LLM, jest procesem czasochłonnym, który nie może blokować głównego wątku serwera Hypertext Transfer Protocol (HTTP).

- Celery[29]: Jest to rozproszony system kolejkowy, pozwalający na definiowanie zadań, które mogą być wykonywane przez wiele procesów roboczych równolegle.
- Redis[23]: Pełni rolę brokera wiadomości, przechowując informacje o zadaniach do wykonania oraz ich wynikach.

2.5 Konteneryzacja: Docker

Zastosowanie konteneryzacji było wymogiem нефункциональным, mającym na celu zapewnienie powtarzalności środowiska uruchomieniowego [2]. Definicja środowiska w pliku `docker-compose.yml` pozwala na uruchomienie całego stosu (Backend, Frontend, Baza Danych, Redis, Worker, LLM Server) jedną komendą, eliminując problem niezgodności w działaniu na różnych maszynach deweloperskich.

Rozdział 3

Analiza wymagań i funkcjonalność systemu

Proces wytwarzania oprogramowania wymaga precyzyjnego zdefiniowania założeń projektowych. W przypadku systemów wykorzystujących generatywną sztuczną inteligencję, kluczowe jest wyznaczenie granic między deterministycznym działaniem algorytmów a probabilistyczną naturą modeli językowych. Poniższy rozdział definiuje specyfikację funkcjonalną i нефункциональную projektowanego systemu, stanowiącą fundament dla implementacji architektury mikroserwisowej opisanej w kolejnych częściach pracy.

3.1 Wymagania funkcjonalne

Głównym celem systemu jest umożliwienie użytkownikom generowania syntetycznych, relacyjnych zbiorów danych o wysokim stopniu realizmu, z wykorzystaniem hybrydowego silnika łączącego metody regułowe i modele LLM.

System powinien udostępniać graficzny interfejs do modelowania struktury bazy danych. Użytkownik powinien mieć możliwość definiowania tabel, kolumn oraz typów danych, wybierając spośród trzech głównych generatorów:

- Generator deterministyczny: Powinien służyć do szybkiego tworzenia standardowych danych o przewidywalnej strukturze (np. imiona, adresy, daty, identyfikatory Universally Unique Identifier (UUID)).
- Generator kontekstowy: Powinien pozwalać na zdefiniowanie szablonu promptu (np. „Napisz e-mail reklamacyjny dotyczący zamówienia {order_id}”), który następnie zostanie przetworzony przez model językowy w celu wygenerowania unikalnej treści.
- Generator dystrybucyjny: Powinien umożliwiać zdefiniowanie ważonych list wartości (np. statystyki zamówień z określonym prawdopodobieństwem wystąpienia), aby odzwierciedlić rzeczywisty rozkład danych.

Zachowanie integralności relacyjnej [27]

Krytyczną funkcjonalnością systemu jest automatyczne zarządzanie relacjami między tabelami. Aplikacja powinna pozwalać na definiowanie kluczy obcych [28], a silnik generujący ma za zadanie automatycznie budować graf zależności i ustalać topologiczną kolejność generowania danych. Mechanizm ten musi gwarantować, że rekordy w tabelach podrzędnych będą zawsze odwoływać się do istniejących rekordów w tabelach nadrzędnych, eliminując błędy spójności.

Orkiestracja zadań generowania

Ze względu na przewidywaną czasochłonność operacji z użyciem LLM, system powinien umożliwiać asynchroniczne uruchamianie procesów generowania. Użytkownik powinien mieć możliwość zlecenia zadania, które trafi do kolejki przetwarzania, a system musi zapewniać mechanizm podglądu postępu prac w czasie rzeczywistym.

Bezpośrednia integracja z bazami danych

Poza standardowym eksportem do plików płaskich (JSON, CSV) czy eksportu do poleceń INSERT, system powinien oferować funkcję bezpośredniego przesyłania do zewnętrznych baz danych. Aplikacja powinna obsługiwać połączenia z najpopularniejszymi silnikami SQL (PostgreSQL, MySQL, MSSQL, Oracle) i wykonywać operacje typu batch insert. Pozwoli to na natychmiastowe wykorzystanie wygenerowanych danych w środowiskach testowych lub deweloperskich bez konieczności ręcznego importu plików.

Zarządzanie projektami i szablonami

Zalogowani użytkownicy powinni mieć możliwość zapisywania zdefiniowanych schematów w chmurze, co pozwoli na ich późniejszą edycję i ponowne wykorzystanie. System powinien również wspierać import i eksport konfiguracji, ułatwiając współdzielenie schematów w zespołach deweloperskich.

3.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają atrybuty jakościowe systemu, które są niezbędne do jego wdrożenia w środowisku produkcyjnym, ze szczególnym naciskiem na wydajność, bezpieczeństwo i skalowalność.

Architektura asynchroniczna i wydajność

System musi być zdolny do obsługi długotrwałych procesów generowania danych bez blokowania interfejsu użytkownika. W tym celu należy zastosować architekturę opartą na kolejce komunikatów, która odseparuje warstwę API od warstwy wykonawczej. Rozwiązanie to powinno zapewniać stabilność

działania nawet przy generowaniu zbiorów liczących dziesiątki tysięcy rekordów, a także odporność na chwilowe błędy dostępności API zewnętrznych.

Prywatność danych i obsługa modeli lokalnych

W odpowiedzi na wymogi bezpieczeństwa, system powinien zapewniać możliwość działania w trybie całkowicie odciętym od sieci zewnętrznej [32]. Wymagane jest umożliwienie integracji z lokalnymi modelami LLM, co zagwarantuje, że żadne dane ani prompty nie opuszczą infrastruktury wewnętrznej użytkownika.

Skalowalność horyzontalna

System powinien być zaprojektowany z myślą o skalowaniu horyzontalnym, wykorzystując konteneryzację. Pozwoli to na łatwe dostosowanie zasobów obliczeniowych do rosnącego zapotrzebowania, poprzez uruchamianie dodatkowych instancji mikroservisów w środowiskach chmurowych lub klastrach kontenerowych.

Bezpieczeństwo dostępu

Dostęp do zasobów systemu musi być chroniony mechanizmem uwierzytelniania. Hasła użytkowników powinny być przechowywane w bazie danych wyłącznie w formie zaszyfrowanej, a cała komunikacja między komponentami systemu oraz z zewnętrznymi bazami danych powinna odbywać się przy użyciu bezpiecznych protokołów.

Rozszerzalność i modularność

Architektura systemu powinna zostać zaprojektowana w sposób modułowy. Ma to na celu umożliwienie łatwego dodawania nowych konektorów baz danych oraz integrację z nowymi dostawcami modeli AI w przyszłości, bez konieczności gruntownej przebudowy rdzenia aplikacji.

Rozdział 4

Metodyka pracy

Rozdział ten poświęcony jest metodyce pracy przyjętej podczas realizacji niniejszej pracy dyplomowej. Znajdują się w nim informacje o narzędziach wspomagających proces twórczy, ze szczególnym uwzględnieniem wykorzystania sztucznej inteligencji, a także wyszczególnienie samodzielnego wkładu autora. Ma to na celu transparentne przedstawienie sposobu, w jaki powstał opisany system oraz rzetelne oddzielenie zautomatyzowanej pomocy od pracy własnej dyplomanta.

4.1 Wykorzystanie wsparcia sztucznej inteligencji

Złożoność oraz rozległy zakres technologiczny projektu - obejmujący zarówno tworzenie interfejsu użytkownika, jak i logiki po stronie serwera - skłoniły dyplomanta do zastosowania nowoczesnych narzędzi opartych na sztucznej inteligencji w procesie programistycznym. Wsparcie to obejmowało korzystanie z tradycyjnych asystentów AI, jak również autonomicznych agentów AI wykonujących powierzone sekwencje zadań technicznych (takich jak analiza kodu w wielu plikach nawzajem). Podczas tworzenia wykorzystywane były w szczególności modele z rodziny Gemini (wersje 3 oraz 3.1 Pro).

Zastosowanie tych technologii pozwoliło na przyspieszenie powtarzalnych operacji i szybsze odnajdywanie specyficznych błędów w kodzie, w związku z czym modele te pełniły funkcję wirtualnego asystenta. Głównymi obszarami, w których posłużono się generatywną sztuczną inteligencją, były:

- Generowanie kodu pomocniczego [33]: Zautomatyzowanie tworzenia powtarzalnych i nużących struktur, deklaracji podstawowych widoków frontendowych czy bazowych szablonów dla plików konfiguracyjnych.
- Debugowanie: Skonsultowanie trudnych do zinterpretowania logów, ostrzeżeń z bibliotek czy błędów pojawiających się w konsoli, co znacząco zmniejszyło czas potrzebny na lokalizację usterek.
- Konsultacje technologiczne i dokumentacyjne: Szybkie odszukiwanie właściwych metod w nowych bibliotekach czy przypominanie rzadko używanych fragmentów składni.

4.2 Wkład autorski i decyzyjność

Mimo wsparcia ze strony narzędzi AI, należy podkreślić, że stanowiły one wyłącznie rozwiązanie wspomagające. Cały system w swoim ostatecznym kształcie jest wynikiem samodzielnej pracy koncepcyjnej, badawczej i programistycznej dyplomanta.

Do kluczowych zadań inżynierskich, zrealizowanych całkowicie samodzielnie przez autora, należą:

- Projektowanie architektury: Samodzielne zaprojektowanie podziału na logiczne warstwy usług, wybór mechanizmów komunikacji między mikroserwisami (Representational State Transfer (REST) API, WebSocket) oraz decyzyjność co do stosu technologicznego (FastAPI, Redis, Celery, React, baza PostgreSQL).
- Implementacja silnika generowania danych: Zaprojektowanie rdzenia logiki backendowej odpowiadającej za walidację oraz generację syntetycznych danych. Wyróżnia się tu m.in. implementację algorytmu sortowania topologicznego rozwiązującego zależności pomiędzy rozbudowanymi relacyjnie tabelami baz danych.
- Projekt interfejsu i interakcji użytkownika (User Interface (UI)/User Experience (UX)): Opracowanie unikalnej koncepcji użytej aplikacji webowej i wizualizacji układu paneli roboczych, która musiała zapewniać ergonomię obsługi.
- Integracja systemu i ocena kodu: Każdy fragment kodu zaproponowany przez narzędzia AI weryfikowany był przez autora pod kątem poprawności i zgodności z ogólną koncepcją systemu.

Wykorzystanie nowoczesnych asystentów programistycznych znacząco odciążało dyplomanta z najbardziej mechanicznych oraz trywialnych aspektów pisania kodu oprogramowania. Zaoszczędzony w ten sposób czas pozwolił na znacznie głębsze skupienie się na meritum pracy: rozwiązywaniu problemów architektonicznych, a także dopracowywaniu detali implementacyjnych.

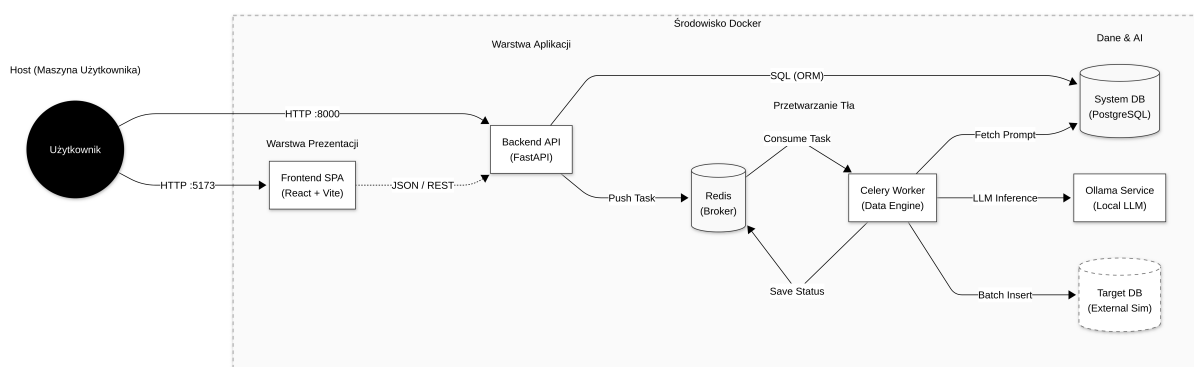
Rozdział 5

Implementacja

W niniejszym rozdziale przedstawiono szczegóły implementacyjne zrealizowanego systemu generatora danych syntetycznych. Opisano architekturę rozwiązania, zastosowane technologie oraz kluczowe komponenty zarówno po stronie serwera (backend), jak i klienta (frontend). Przedstawiono również sposób konfiguracji środowiska uruchomieniowego oraz proces wdrażania aplikacji przy użyciu konteneryzacji.

5.1 Architektura systemu

System został zaprojektowany w architekturze klient-serwer, z wyraźnym podziałem na warstwę prezentacji (frontend) oraz warstwę logiki biznesowej i dostępu do danych (backend). Komunikacja między tymi warstwami odbywa się za pośrednictwem interfejsu REST API [5]. Całość systemu została skonteneryzowana przy użyciu platformy Docker, co zapewnia łatwość wdrożenia i spójność środowiska uruchomieniowego. Poniżej przedstawiono schemat architektury systemu (Rysunek 1).



Rysunek 1. Schemat architektury systemu

Główne elementy systemu to:

- Backend API: Serwer aplikacji odpowiedzialny za logikę biznesową, autentykację użytkowników oraz zarządzanie procesem generowania danych.

- Frontend: Aplikacja internetowa umożliwiająca użytkownikom interakcję z systemem, definiowanie schematów danych oraz podgląd wyników.
- Baza danych (PostgreSQL): Przechowuje informacje o użytkownikach, projektach oraz historię zadań.
- Kolejka zadań (Redis): Pośredniczy w przekazywaniu zadań generowania danych do procesów roboczych (workerów), co pozwala na asynchroniczne przetwarzanie długotrwałych operacji.
- Worker (Celery): Proces odpowiedzialny za właściwe generowanie danych na podstawie zdefiniowanych reguł.
- Ollama (LLM): Lokalny serwer modeli językowych, wykorzystywany do generowania danych wymagających kreatywności i kontekstu [21].

5.2 Implementacja backendowa

Warstwa backendowa została zaimplementowana w języku Python, z wykorzystaniem frameworka FastAPI. Wybór ten podyktowany był wysoką wydajnością FastAPI, wbudowaną obsługą asynchroniczności oraz automatycznym generowaniem dokumentacji API (Swagger UI).

Struktura projektu

Kod źródłowy backendu został podzielony na moduły zgodnie z zasadą pojedynczej odpowiedzialności:

- `main.py`: Punkt wejścia aplikacji, konfiguracja serwera i definicja tras API.
- `models.py`: Modele danych Pydantic służące do walidacji żądań i odpowiedzi API.
- `database.py`: Konfiguracja połączenia z bazą danych przy użyciu biblioteki SQLAlchemy.
- `engine.py`: Główny silnik generowania danych.
- `tasks.py`: Definicje zadań asynchronicznych dla Celery.
- `auth.py`: Logika uwierzytelniania i autoryzacji (OAuth2, haszowanie haseł).

Silnik generowania danych (Data Engine)

Sercem systemu jest klasa `DataEngine` znajdująca się w pliku `engine.py`. Odpowiada ona za interpretację schematu danych zdefiniowanego przez użytkownika i fizyczne wygenerowanie wartości dla poszczególnych pól.

Kluczowym wyzwaniem implementacyjnym było obsłużenie zależności między tabelami (klucze obce). W tym celu zaimplementowano metodę `_resolve_generation_order`, która analizuje graf zależności między tabelami i ustala poprawną kolejność generowania. Algorytm ten zapewnia, że dane do tabeli nadrzędnej są generowane przed danymi do tabeli podrzędnej.

Generowanie wartości dla poszczególnych pól realizowane jest przez dedykowane metody:

- `_generate_template_value`: Pozwala na tworzenie wartości pochodnych przy użyciu szablonów Jinja2, bazując na innych polach w tej samej tabeli.

- `_generate_regex_value`: Generuje ciągi znaków pasujące do podanego wyrażenia regularnego, wykorzystując bibliotekę `rstr`.
- `_generate_timestamp_value`: Generuje losowe daty i czasy w określonym przedziale.
- `_generate_integer_or_float_value`: Generuje liczby całkowite lub zmiennoprzecinkowe w zadanym zakresie.
- `_generate_boolean_value`: Zwraca wartość logiczną prawda/fałsz z określonym prawdopodobieństwem.
- `_generate_faker_value`: Wykorzystuje bibliotekę `Faker` do tworzenia realistycznych danych (imiona, adresy, emaile).
- `_generate_distribution_value`: Losuje wartość z podanej listy opcji z uwzględnieniem rozkładu prawdopodobieństwa.
- `_generate_foreign_key_value`: Rozwiązuje relacje między tabelami, pobierając identyfikator z innej, wcześniej wygenerowanej tabeli.
- `_generate_llm_value`: Komunikuje się z modelem językowym (przez OpenAI API lub lokalną instancję Ollama) w celu wygenerowania danych kontekstowych.

Przetwarzanie asynchroniczne

Ze względu na potencjalnie długi czas generowania dużych zbiorów danych, proces ten został odseparowany od głównego wątku obsługi żądań HTTP. Wykorzystano w tym celu bibliotekę Celery oraz bazę Redis jako brokera wiadomości.

Użytkownik, zlecając generowanie danych, wysyła żądanie do endpointu `/generate/async`. System tworzy nowe zadanie, zwraca identyfikator `job_id` i kończy obsługę żądania HTTP. Właściwe generowanie odbywa się w tle przez proces Workera. Status zadania jest monitorowany przez klienta w czasie rzeczywistym za pomocą protokołu WebSocket (endpoint `/ws/jobs/{job_id}`), co pozwala na wyświetlanie paska postępu.

Bezpieczeństwo

System implementuje mechanizm uwierzytelniania oparty na tokenach JSON Web Token (JWT) [9]. Hasła użytkowników są bezpiecznie przechowywane w bazie danych w formie zahasowanej (przy użyciu algorytmu `bcrypt`). Każdy chroniony endpoint API weryfikuje ważność tokenu przed przetworzeniem żądania, co realizowane jest za pomocą mechanizmu Dependency Injection w FastAPI.

5.3 Implementacja frontendowa

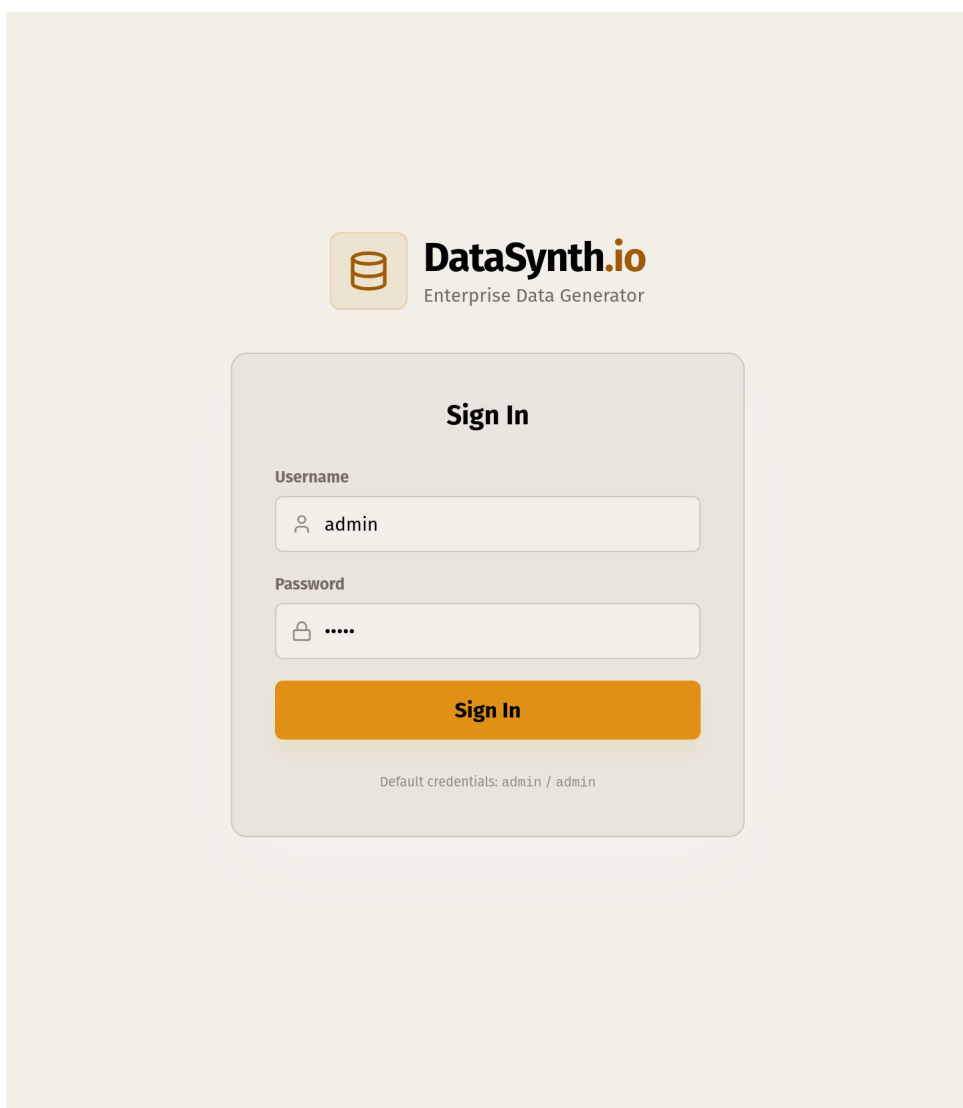
Warstwa prezentacji została zrealizowana jako aplikacja typu Single Page Application (SPA) przy użyciu biblioteki React. Do budowania projektu wykorzystano narzędzie Vite, a za stylowanie odpowiada framework Tailwind Cascading Style Sheets (CSS).

Szczegółowy opis interfejsu

W dalszej części tego rozdziału przedstawiono kluczowe elementy interfejsu graficznego aplikacji. Interfejs został zaprojektowany w sposób minimalistyczny, z wykorzystaniem ciemnego motywu, aby zapewnić komfort pracy przy długotrwałym użytkowaniu. Aplikacja jest w języku angielskim, co jest standardem w narzędziach deweloperskich.

Ekran logowania

Dostęp do systemu jest chroniony i wymaga uwierzytelnienia.



Rysunek 2. Ekran logowania do systemu

Na rysunku 2 przedstawiono widok logowania. Użytkownik musi podać nazwę użytkownika oraz hasło. System komunikuje się z API w celu weryfikacji poświadczeń i w przypadku sukcesu otrzymuje

token JWT, który jest zapisywany w pamięci przeglądarki. Interfejs obsługuje również wyświetlanie komunikatów o błędach w przypadku niepoprawnych danych.

Główny widok aplikacji

Po zalogowaniu użytkownik dostaje dostęp do głównego widoku aplikacji, który stanowi centrum tworzenia schematu bazy danych. Interfejs został zaprojektowany z myślą o ergonomii, dzieląc ekran na dwie główne sekcje: lewy panel konfiguracyjny (budowanie schematu) oraz prawy panel wyników (podgląd i eksport). Opis obu części znajduje się poniżej.

Lewy panel zawiera zestaw narzędzi niezbędnych do zdefiniowania struktury zbioru danych. Składa się on z następujących kluczowych modułów:

1. Pasek narzędzi

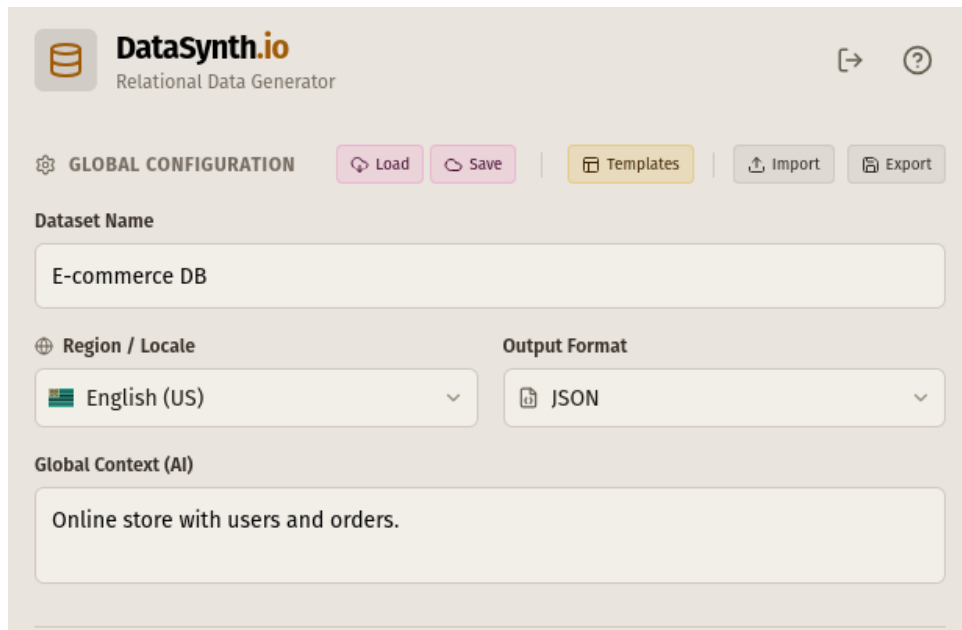
W górnej części panelu znajduje się pasek narzędzi służący do zarządzania stanem całego projektu. Użytkownik ma tutaj dostęp do następujących funkcjonalności:

- Zarządzanie projektami: Przyciski umożliwiające zapisywanie i wczytywanie projektów z bazy danych.
- Szablony (Templates): Dostęp do gotowych, predefiniowanych schematów danych.
- Import/Eksport: Możliwość zapisania schematu do lokalnego pliku JSON oraz jego ponownego wczytania.
- Wczytywanie przykładowych danych: Opcja pozwalająca na szybkie załadowanie przykładowego schematu (np. „E-commerce”), co jest szczególnie przydatne dla nowych użytkowników.

2. Konfiguracja globalna (Global Configuration)

Poniżej paska narzędzi znajdują się parametry definiujące ogólne właściwości generowanego zbioru danych:

- Nazwa zbioru: Pole tekstowe definiujące nazwę zadania, która później służy jako nazwa pliku wynikowego.
- Lokalizacja (Locale): Rozwijana lista pozwalająca wybrać krąg kulturowy generowanych danych (np. en_US, pl_PL, de_DE). Wybór ten wpływa na formatowanie dat, walut, numerów telefonów oraz język generowanych tekstów przez bibliotekę Faker.
- Format wyjściowy: Pozwala zdefiniować, w jakiej postaci mają zostać zwrócone dane (JSON, CSV spakowany w ZIP, plik SQL).
- Globalny kontekst AI: Pole tekstowe, w którym użytkownik może opisać ogólny charakter generowanych danych (np. „System medyczny dla szpitala wojewódzkiego”). Kontekst ten jest „doklejany” do promptów wysyłanych do modeli LLM, co pozwala na zachowanie spójności tematycznej wszystkich generowanych pól.



The screenshot shows the 'GLOBAL CONFIGURATION' section of the DataSynth.io interface. At the top, there's a header with the logo and navigation links. Below it, a toolbar contains 'Load', 'Save', 'Templates', 'Import', and 'Export' buttons. The main configuration area includes a 'Dataset Name' field with the value 'E-commerce DB'. Below this are two dropdown menus: 'Region / Locale' set to 'English (US)' and 'Output Format' set to 'JSON'. At the bottom, a 'Global Context (AI)' text area contains the description 'Online store with users and orders.'

Rysunek 3. Formularz konfiguracji globalnej i pasek narzędzi

3. Menedżer tabel (Table Manager)

Poniżej konfiguracji znajduje się lista zdefiniowanych tabel. Moduł ten pozwala na:

- Tworzenie struktur relacyjnych: Użytkownik może dodawać dowolną liczbę tabel, co pozwala na modelowanie złożonych relacji bazodanowych.
- Kontrolę wolumenu danych: Każda tabela posiada niezależny licznik wierszy, co pozwala np. wygenerować 100 użytkowników i 5000 zamówień.
- Przeglądanie tabel: Kliknięcie w nagłówek tabeli aktywuje ją, zmieniając kontekst dla listy pól i edytora.



The screenshot displays the 'DATABASE TABLES (3)' section. It features a list of three tables: 'users', 'orders', and 'items'. Each table entry includes a table icon, the name, and a row count indicator showing '# ROWS | 10'. The 'items' table is currently selected, indicated by an orange border. A '+ Add Table' button is located in the top right corner of the table list area.

Rysunek 4. Menedżer tabel

4. Kreator pól (Schema Builder)

Kluczowym elementem interfejsu jest formularz dodawania nowych pól do tabeli.

The screenshot shows the 'SCHEMA BUILDER' interface. It features a form with the following elements:

- FIELD NAME:** A text input containing 'e.g. user_id'.
- TYPE:** A dropdown menu currently showing 'Faker Library'.
- Faker Method:** A dashed box containing the text 'UUID (Unique ID)'.
- Unique Value:** A checkbox that is currently checked.
- Field Preview:** A section showing 'id' with 'FAKER' and 'UNIQ' tags.
- Run Generation:** A large orange button with a play icon and the text 'Run Generation'.
- Standard Data List:** A sidebar on the right listing various data types: 'AI Generation' (GPT-4o creative content), 'Derived / Logic' (Combine fields (Jinja2)), 'Distribution' (Weighted random options), 'Faker Library' (Real-world data (names, address)), 'Number' (Range of numbers), 'Boolean' (True / False flag), and 'Timestamp' (Date & Time range).

Rysunek 5. Formularz definiowania nowego pola

Moduł ten (Rysunek 5) dynamicznie dostosowuje dostępne parametry w zależności od wybranego typu danych. Użytkownik może:

- Wybrać typ generatora (np. Faker, AI/LLM, Integer, Foreign Key).
- Skonfigurować parametry specyficzne (np. zakres liczb, prompt dla AI, tabelę docelową dla klucza obcego).
- Oznaczyć pole jako unikalne (Unique), co wymusi generowanie niepowtarzalnych wartości.

Na rysunkach 6 oraz 7 przedstawiono przykładowe konfiguracje dla generatora AI oraz generatora z rozkładem prawdopodobieństwa.

The image shows a web interface titled "SCHEMA BUILDER". It contains a form for defining a new field. At the top, there are two sections: "FIELD NAME" with a text input containing "e.g. user_id", and "TYPE" with a dropdown menu showing "AI Generation" with a green icon. Below these is a dashed box containing several options: "Provider" with a dropdown set to "OpenAI (Cloud)", "Model" with a dropdown set to "GPT-4o Mini (Fast)", a "Prompt Template" text area with "e.g. Generate a name...", an "Insert Variable" section with a "{id}" button, a "Temperature" slider set to 1.0 (labeled "Creative (1.0)"), and a "Show Advanced Parameters" link. At the bottom of the form, there is a "Unique Value" checkbox and a purple "+ Add Field" button.

Rysunek 6. Formularz definiowania nowego pola z wykorzystaniem AI

The screenshot shows the 'SCHEMA BUILDER' interface. At the top, there's a 'FIELD NAME' input field with the placeholder 'e.g. user_id' and a 'TYPE' dropdown menu set to 'Distribution'. Below this, a dashed box contains a 'Weighted Random' section with the instruction 'Define options and their relative probability weights.' Inside this section, there's a table with two columns: 'Option Label' and 'Weight'. It lists 'Option A' and 'Option B', both with a weight of '50 %'. A 'Total: 100.00' indicator is shown at the bottom of the table. To the right of the table is a '+ Add Option' button. Below the dashed box, there's a 'Unique Value' checkbox and a '+ Add Field' button.

Option Label	Weight
Option A	50 %
Option B	50 %

Total: 100.00

+ Add Option

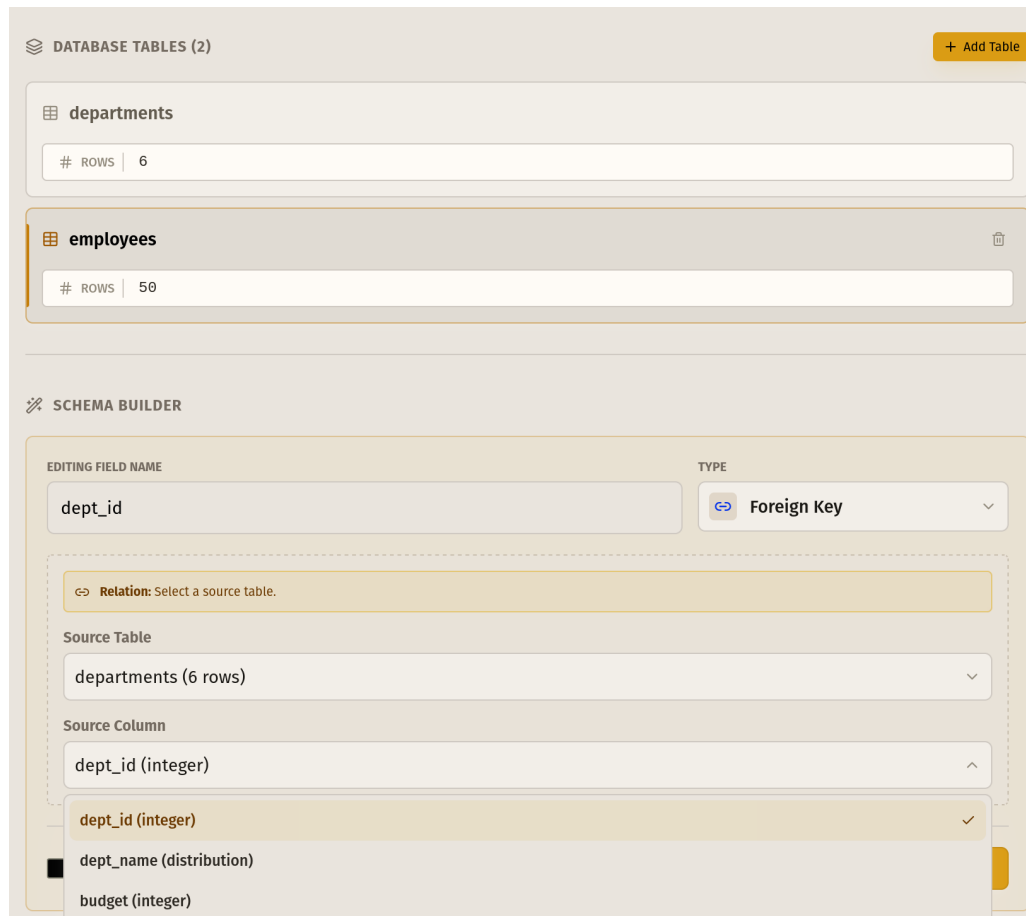
☐ Unique Value

+ Add Field

Rysunek 7. Formularz definiowania nowego pola z wykorzystaniem rozkładu prawdopodobieństwa

5. Lista pól (Field List)

Dolną część panelu zajmuje lista pól dla aktywnej tabeli. Każdy element listy prezentuje nazwę kolumny, typ generatora (oznaczony kolorowym badge'm dla łatwej identyfikacji) oraz ewentualne flagi (np. UNIQ dla wartości unikalnych). Moduł umożliwia również szybką edycję parametrów oraz usuwanie poszczególnych pól ze struktury.



Rysunek 8. Interfejs konfigurowania klucza obcego pośród opcji parametrów w panelu bocznym

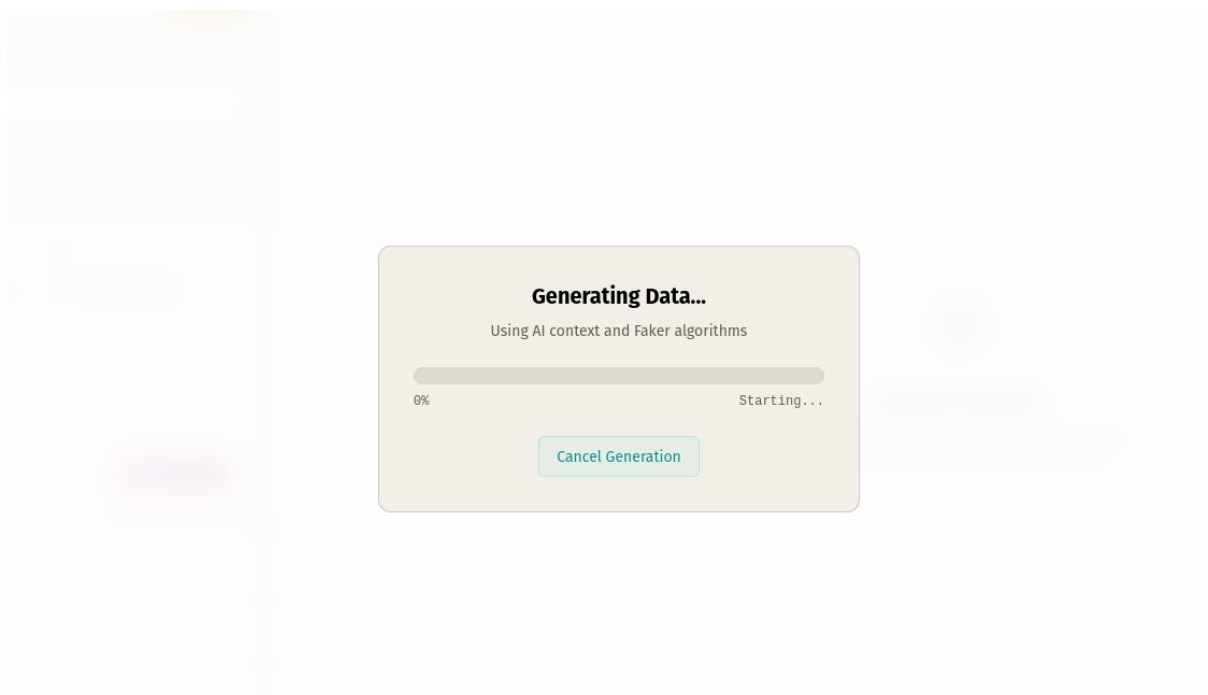
Rysunek 8 przedstawia zrzut ekranu panelu ustawień wybranego pola w kreatorze schematu. Po wskazaniu typu generatora klucza obcego, użytkownikowi automatycznie prezentowane są dedykowane parametry, pozwalające na wygodne wskazanie tabeli nadrzędnej oraz konkretnej kolumny (z wcześniej dodanych przez użytkownika pól). Rozwiązanie to zdejmuje z twórcy schematu ciężar konieczności manualnego wpisywania komend SQL, znacznie ułatwiając sprawne i wizualne prototypowanie relacji między tabelami.



Rysunek 9. Lista pól dla aktywnej tabeli

Generowanie i eksport danych

Proces generowania danych odbywa się asynchronicznie, a postęp jest wizualizowany za pomocą paska postępu.



Rysunek 10. Okno postępu generowania

Po zakończeniu procesu (Rysunek 10), wyniki są prezentowane w panelu wyjściowym. Użytkownik ma wtedy możliwość:

- Przeglądania wygenerowanych danych w formacie tabeli, jak i w formacie JSON bezpośrednio w przeglądarce (wprowadzono ograniczenie do 100 wierszy, aby uniknąć problemów z wydajnością).
- Skopiowania widocznego fragmentu danych do schowka.
- Pobrania danych jako plik JSON, SQL lub pakiet CSV.
- Bezpośredniego eksportu danych do zewnętrznej bazy danych (poprzez podanie parametrów połączenia).

Push to Database

Database Connection String (URI)

postgresql://client:clientpass@target-db:5432/cli

Supported Formats:

PostgreSQL:	postgresql://user:pass@host:5432/db
MySQL:	mysql+pymysql://user:pass@host:3306/db
MSSQL:	mssql+pymssql://user:pass@host:1433/db
Oracle:	oracle+oracledb://user:pass@host:1521/service_name

Test Connection Push Data

Rysunek 11. Okno eksportu danych do bazy danych

Na rysunku 11 przedstawiono formularz umożliwiający bezpośredni eksport danych do zewnętrznej bazy danych, co jest szczególnie przydatne dla deweloperów chcących szybko zasilić testową bazę danymi.

Z racji na zbyt duży rozmiar pliku, zrezygnowano z umieszczania zrzutów ekranu z widokiem wygenerowanych danych w pracy pisemnej.

Stan aplikacji jest zarządzany głównie przy użyciu hooków React (`useState`, `useEffect`). Do obsługi złożonego stanu edytora schematu stworzono dedykowany hook `useSchema`, który centralizuje logikę dodawania, usuwania i edycji tabel oraz pól.

Do komunikacji z backendem wykorzystano bibliotekę `axios`. Zaimplementowano instancję klienta API (w pliku `api.js`), która automatycznie dołącza token autoryzacyjny do każdego żądania oraz obsługuje błędy odpowiedzi serwera (np. automatyczne wylogowanie po wygaśnięciu tokenu).

5.4 Implementacja algorytmów

Algorytm rozwiązywania zależności (sortowanie topologiczne)

Aby zapewnić poprawność relacyjną generowanych danych, niezbędne jest ustalenie odpowiedniej kolejności przetwarzania tabel. Problem ten sprowadza się do posortowania wierzchołków skierowanego grafu zależności w taki sposób, aby dla każdej krawędzi (u, v) wierzchołek u pojawiał się przed wierzchołkiem v .

W systemie zaimplementowano wariant algorytmu Kahna dla sortowania topologicznego [10]. Proces przebiega w następujących krokach:

1. Budowa grafu zależności: Dla każdej tabeli analizowanemu są jej pola. Jeśli pole jest typu `foreign_key`, dodawana jest zależność od tabeli docelowej.
2. Inicjalizacja: Tworzona jest lista tabel „gotowych” (nieposiadających niewygenerowanych zależności).
3. Iteracja: Dopóki lista „gotowych” tabel nie jest pusta:
 - Wybierana jest tabela, dodawana do listy wynikowej.
 - Tabela ta jest usuwana z zależności innych tabel.
 - Jeśli w wyniku usunięcia inna tabela traci wszystkie zależności, trafia na listę „gotowych”.
4. Weryfikacja: Jeśli po zakończeniu algorytmu lista wynikowa jest krótsza niż całkowita liczba tabel, oznacza to wykrycie cyklu, co jest zgłaszane jako błąd.

Integracja z LLM

Generowanie danych przy użyciu LLM odbywa się w metodzie `_generate_llm_value`. Proces ten jest znacznie bardziej złożony niż proste wywołanie API.

Prompt wysyłany do modelu składa się z trzech części:

- System Message: Instruuje model, aby wcielił się w rolę generatora danych fikcyjnych.
- Szablon użytkownika: Szablon zdefiniowany przez użytkownika, np. „Opisz osobę {name} {surname}”. Zmienne w nawiasach klamrowych są podmieniane na wartości wygenerowane w bieżącym wierszu przy użyciu silnika Jinja2.
- Ograniczenia: Dodawane dynamicznie instrukcje, np. wymuszenie unikalności.

Zapewnienie unikalności w modelach probabilistycznych jest wyzwaniem. System stosuje podejście iteracyjne:

```

attempts = 0
while attempts < 10:
    temperature = base_temp + (attempts * 0.1)
    prompt = construct_prompt(template, avoid_values)
    context_injection(prompt, current_row_data)

```

```
value = await call_llm(prompt, temperature)

if is_unique(value):
    return value

attempts += 1
```

Temperatura (temperature) jest hiperparametrem kontrolującym losowość generowanych odpowiedzi LLM. Wzrost temperatury w kolejnych próbach spłaszcza rozkład prawdopodobieństwa kolejnych słów, co zwiększa „kreatywność” modelu i znacząco zmniejsza szansę na wygenerowanie duplikatu.

W celu optymalizacji wydajności generowania danych LLM, zaimplementowano dwufazowy algorytm przetwarzania. W pierwszej fazie silnik generuje wszystkie pola deterministyczne (z parametrem skip_llm=True), budując kontekst każdego wiersza. Następnie zebrane konteksty są przekazywane do metody _generate_llm_batch_value, która dzieli je na podpartie o stałym rozmiarze (SUB_BATCH_SIZE). Dla każdej podpartii formułowany jest zbiorczy prompt, a model jest instruowany, by zwrócił tablicę JSON z odpowiedziami. Podpartie są przetwarzane współbieżnie z ograniczeniem przez asyncio.Semaphore (MAX_CONCURRENT slotów). Szczegóły procesu optymalizacji i doboru parametrów opisano w Sekcji 6.3.

Do wizualizacji postępu w czasie rzeczywistym wykorzystano protokół WebSocket. Po nawiązaniu połączenia pod adresem ws://server:8001/ws/jobs/{job_id}, serwer cyklicznie przesyła ramki JSON o strukturze:

```
{
  "job_id": "9fa2204c-72d6-...",
  "status": "processing", // lub "completed", "failed"
  "progress": 45,         // wartość procentowa 0-100
  "total_rows": 1000,
  "error": null           // ewentualny komunikat błędu
}
```

Frontend (komponent GenerationModal) subskrybuje te wiadomości i na ich podstawie aktualizuje pasek postępu. Po otrzymaniu statusu completed, połączenie jest zamykane, a interfejs przechodzi do widoku wyników.

5.5 Infrastruktura i konteneryzacja

Całe środowisko uruchomieniowe zostało zdefiniowane w pliku docker-compose.yml. Definiuje on następujące serwisy:

- backend: Kontener z aplikacją FastAPI.
- frontend: Kontener serwujący pliki statyczne aplikacji React (lub serwer developerski w trybie dev).

- `worker`: Kontener procesu Celery do zadań w tle.
- `db`: Baza danych PostgreSQL dla aplikacji.
- `redis`: Broker wiadomości dla Celery.
- `target-db`: Dodatkowa instancja PostgreSQL, służąca jako przykładowa docelowa baza danych do bezpośredniego zasilania danymi.
- `ollama`: Usługa serwująca modele LLM.

Dzięki takiemu podejściu, uruchomienie całego systemu sprowadza się do wykonania jednego polecenia `docker compose up`, co znacząco upraszcza proces instalacji i testowania rozwiązania.

Rozdział 6

Weryfikacja systemu, wyzwania i wnioski

Ostatnia część pracy łączy weryfikację poprawności i bezpieczeństwa zaimplementowanego systemu, omówienie napotkanych problemów technicznych wraz z ich rozwiązaniami oraz podsumowanie uzyskanych rezultatów. Takie ujęcie pozwala oddzielić testy funkcjonalne od analizy wydajnościowej — która bezpośrednio wynika z wyzwań związanych z integracją modeli LLM — i zakończyć pracę spójnymi wnioskami.

6.1 Testy poprawności i bezpieczeństwa

Proces weryfikacji poprawności działania systemu został podzielony na testy automatyczne warstwy backendowej, testy manualne interfejsu użytkownika, testy E2E oraz audyt bezpieczeństwa. Takie podejście pozwoliło na wykrycie błędów zarówno na poziomie logicznym, jak i funkcjonalnym.

6.1.1 Testy jednostkowe i integracyjne backendu

Warstwa serwerowa aplikacji, zaimplementowana w języku Python, została pokryta testami automatycznymi przy użyciu frameworka `pytest`. Głównym celem tych testów była weryfikacja poprawności działania poszczególnych endpointów API oraz logiki biznesowej generatora danych.

Testy, zlokalizowane w katalogu `backend/tests`, obejmują następujące scenariusze:

- Weryfikacja autentykacji: Sprawdzenie poprawności generowania i walidacji tokenów JWT. Testy upewniają się, że dostęp do chronionych zasobów bez ważnego tokenu jest blokowany (kod błędu 401 `Unauthorized`).
- Create, Read, Update, Delete (CRUD) projektów: Testowanie operacji tworzenia, odczytu i usuwania projektów. Weryfikowana jest poprawność zapisu struktury JSON do bazy danych PostgreSQL.
- Walidacja schematów Pydantic: Sprawdzenie, czy system poprawnie odrzuca błędne żądania (np. brak wymaganych pól, niepoprawne typy danych).

- Symulacja generowania danych: Uruchamianie silnika generującego dla prostych konfiguracji w celu potwierdzenia, że zwracane dane są zgodne z zadeklarowanym typem.

Przykładowy test weryfikujący działanie endpointu `/generate` wygląda następująco:

```
def test_generate_endpoint(client, auth_headers):
    payload = {
        "config": {"job_name": "Test Job", "rows": 10},
        "tables": [...]
    }
    response = client.post("/generate", json=payload, headers=auth_headers)
    assert response.status_code == 200
    data = response.json()
    assert "status" in data
    assert data["status"] == "success"
```

6.1.2 Testy manualne interfejsu użytkownika

Ze względu na dynamiczny charakter interfejsu, duży nacisk położono na testy manualne. Scenariusze testowe obejmowały pełne ścieżki użytkownika (ang. *User Journeys*).

Scenariusz 1: Utworzenie schematu sklepu internetowego

Celem tego testu było sprawdzenie integralności relacyjnej generowanych danych.

1. Utworzenie tabeli `users` z polami `id`, `name`, `email`.
2. Utworzenie tabeli `orders` z polem `user_id` jako klucz obcy do tabeli `users`.
3. Uruchomienie generowania dla 100 użytkowników i 500 zamówień.
4. Weryfikacja: Sprawdzenie, czy każde zamówienie przypisane jest do istniejącego użytkownika.
Test zakończył się sukcesem — silnik poprawnie rozwiązał zależności w 100% przypadków.

Scenariusz 2: Integracja z LLM

Testowano zachowanie systemu przy generowaniu pól typu `llm` oraz poprawność obsługi asynchronicznego postępu zadania.

1. Zdefiniowanie pola typu `llm` z promptem generującym opis produktu.
2. Ustawienie liczby wierszy na 50.
3. Obserwacja: Interfejs pozostawał responsywny podczas oczekiwania na odpowiedź z API OpenAI, a pasek postępu aktualizował się w czasie rzeczywistym. Po zakończeniu zadania wygenerowano kompletny zestaw rekordów zgodny ze schematem. Szczegóły pomiarów wydajnościowych oraz optymalizacji opisano w Sekcji 6.3.

6.1.3 Testy End-to-End (E2E)

W celu weryfikacji integralności całego systemu wprowadzono testy typu E2E, symulujące rzeczywiste scenariusze użytkowania — od interfejsu w przeglądarce, przez warstwę API, aż po logikę biznesową i bazy danych.

Do realizacji testów E2E wybrano framework Playwright. Jest on nowoczesnym narzędziem umożliwiającym automatyzację testów w wielu przeglądarkach (Chromium, Firefox, WebKit) oraz zapewniającym wysoką niezawodność dzięki mechanizmom automatycznego oczekiwania na elementy interfejsu.

Zaimplementowany scenariusz testowy pokrywa kluczową ścieżkę krytyczną [34] użytkownika:

1. Autoryzacja: Scenariusz rozpoczyna się od wejścia na stronę logowania. Automat testowy wprowadza dane uwierzytelniające administratora i weryfikuje poprawne przekierowanie do głównego widoku aplikacji.
2. Definicja schematu: Wirtualny użytkownik dodaje nową tabelę o nazwie „users” oraz konfiguruje jej strukturę, dodając kolumnę „name”. Test weryfikuje poprawność interakcji z formularzami oraz dynamiczne aktualizacje interfejsu.
3. Generowanie danych: Następuje uruchomienie procesu generowania danych poprzez kliknięcie przycisku „Run Generation”. Test monitoruje asynchroniczny proces, oczekując na komunikaty o postępie przekazywane przez WebSocket.
4. Weryfikacja wyników: Ostatnim krokiem jest potwierdzenie zakończenia zadania sukcesem (pojawienie się komunikatu „Data generated!”) oraz sprawdzenie dostępności przycisku pobierania wyników.

Dzięki zastosowaniu symulowania odpowiedzi backendu w wybranych testach możliwe jest izolowane testowanie warstwy frontendowej, co przyspiesza proces deweloperski i uniezależnia testy interfejsu od dostępności zewnętrznych usług czy modeli LLM.

6.1.4 Testy bezpieczeństwa

W celu zapewnienia wysokiego poziomu bezpieczeństwa przeprowadzono serię testów z wykorzystaniem narzędzi klasy Software Composition Analysis (SCA) oraz Static Application Security Testing (SAST).

Do identyfikacji znanych podatności w wykorzystywanych bibliotekach użyto narzędzi `npm audit` (dla warstwy frontendowej) oraz `pip-audit` (dla warstwy backendowej).

- Frontend: Analiza początkowo wykazała obecność podatności w bibliotece `axios` (wersja $\leq 1.3.4$). W ramach działań naprawczych zaktualizowano bibliotekę do najnowszej stabilnej wersji, eliminując zidentyfikowane zagrożenia.
- Backend: Audyt zależności Pythonowych nie wykazał krytycznych podatności w głównych bibliotekach produkcyjnych (`FastAPI`, `Uvicorn`, `SQLAlchemy`).

Kod źródłowy części serwerowej został poddany analizie przy użyciu narzędzia Semgrep [26]. Wykryto potencjalne zagrożenie w konfiguracji mechanizmu Cross-Origin Resource Sharing (CORS), polegające na zbyt szerokim uprawnieniu dostępu (`allow_origins=["*"]`). Problem ten został rozwiązany poprzez wprowadzenie konfiguracji opartej na zmiennych środowiskowych, co pozwala na ściśle zdefiniowanie zaufanych domen w środowisku produkcyjnym:

```
allow_origins=os.getenv("ALLOWED_ORIGINS").split(",")
```

6.2 Wyzwania implementacyjne

Stworzenie aplikacji łączącej konwencjonalne generowanie danych z wykorzystaniem modeli sztucznej inteligencji wiązało się z szeregiem problemów technicznych. Poniżej opisano najistotniejsze z nich oraz przyjęte rozwiązania — z wyjątkiem kwestii wydajności generowania przez LLM, które ze względu na swój zasięg omówiono osobno w kolejnej sekcji.

6.2.1 Informowanie użytkownika o postępie

Jednym z istotnych problemów na początku prac było płynne wyświetlanie paska postępu generowania na stronie internetowej, podczas gdy cały proces odbywał się w tle na serwerze.

Standardowe zapytania HTTP nie sprawdzają się najlepiej w tym zastosowaniu, ponieważ przeglądarka musiałaby nieustannie pytać serwer o aktualny status. Przy większej liczbie użytkowników spowodowałoby to znaczne przeciążenie sieci. Zdecydowano się na wykorzystanie protokołu WebSocket. W backendzie utworzono specjalny endpoint `/ws/jobs/{job_id}`, do którego podłącza się aplikacja. Program działający w tle regularnie zapisuje swój postęp do pamięci współdzielonej, a połączenie WebSocket odczytuje te dane co pół sekundy i przesyła na bieżąco do przeglądarki. Dzięki temu interfejs działa responsywnie i płynnie aktualizuje pasek postępu.

6.2.2 Wymuszanie unikalności danych z modelu LLM

Modele językowe charakteryzują się niedeterministycznym działaniem, co stanowiło wyzwanie w przypadku generowania kolumn, dla których zaznaczono wymóg unikalności (np. dla numerów seryjnych generowanych przez AI).

Podczas testów zauważono, że przy ustawieniu wysokiej „temperatury” (parametr `temperature`) model często tracił kontekst i nie trzymał się narzuconego formatu. Z kolei przy niskiej temperaturze wykazywał tendencję do generowania tych samych słów i schematów. Aby zapobiec powstawaniu duplikatów, zaimplementowano własny mechanizm weryfikacji:

1. Aplikacja zapisuje w pamięci listę wygenerowanych wartości dla danej unikalnej kolumny.
2. Przy każdej nowej odpowiedzi od sztucznej inteligencji następuje sprawdzenie, czy dana wartość nie została już wykorzystana.

3. W przypadku wykrycia duplikatu system ponawia zapytanie (maksymalnie do 10 razy). Do promptu doklejana jest dodatkowa instrukcja: `CONSTRAINT: Value MUST be unique. DO NOT use: [lista wykorzystanych wartości]`.
4. Aby pomóc modelowi w znalezieniu nowej odpowiedzi, przy kolejnych próbach aplikacja delikatnie zwiększa parametr `temperature (+0.1)`, co zachęca algorytm do większej różnorodności w doborach słów.

6.2.3 Zależności cykliczne

Aplikacja sortuje tabele przed generowaniem, aby zapewnić odpowiednią kolejność ich tworzenia (np. najpierw użytkownicy, potem ich opinie). Problem pojawiał się, gdy Tabela A wymagała klucza z Tabeli B, a Tabela B jednocześnie wymagała klucza z Tabeli A.

Tworzył się wówczas tzw. cykl zależności, który zapętliał program. Na obecnym etapie przyjęto założenie, że użytkownik musi świadomie projektować strukturę bazy i unikać takich sytuacji w interfejsie. W przyszłości planowane jest dodanie dodatkowej walidacji, która blokowałaby zapis schematu w przypadku wykrycia „błędnego koła”.

6.2.4 Odporność na błędy łączności z API

Komunikacja z zewnętrznymi interfejsami, takimi jak API usługi OpenAI, niesie ze sobą ryzyko sporadycznej utraty połączenia sieciowego lub przeciążeń po stronie dostawcy chmury. W kodzie serwera uodporniono wywołania API za pomocą precyzyjnych bloków wychwytywania wyjątków `try-except`. W przypadku wystąpienia *timeoutu* (braku odpowiedzi sieciowej dla konkretnego zapytania) system nie odrzuca całego procesu po wielu godzinach pracy. Kod zachowuje odpowiedni tekst błędu (np. `Connection error`), wpisując go bezpiecznie w odpowiednią komórkę bazy danych tabeli. Pozwala to na płynne kontynuowanie generowania kolejnego wiersza, uniemożliwiając restart skryptu i pozostawiając operatorowi wgląd w omyłkowy element poprzez odnalezienie komunikatu błędu w docelowym pliku Excel bądź CSV.

6.3 Wydajność i optymalizacja generowania

Największym wyzwaniem dla wydajności systemu okazało się generowanie opisów przez modele LLM. Początkowa wersja kodu zakładała przetwarzanie wierszy pojedynczo: program tworzył wiersz, wysyłał zapytanie do serwera Ollama, czekał na odpowiedź i dopiero wtedy przechodził do kolejnego rekordu.

Pomiary wykazały, że wygenerowanie 100 recenzji produktów przez model Llama 3 zajmowało blisko 130 sekund. Prawie 95% tego czasu stanowiło oczekiwanie na odpowiedź sztucznej inteligencji, podczas gdy zwykłe dane tekstowe (np. nazwiska z biblioteki Faker) generowały się w ułamki sekund. Czas oczekiwania na jeden rekord wynosił około 1,3 sekundy, a procesor oraz karta graficzna przez większość tego czasu nie były w pełni obciążone.

6.3.1 Algorytm dwufazowy i przetwarzanie wsadowe

W celu lepszego wykorzystania zasobów obliczeniowych wprowadzono algorytm dwufazowy:

1. W pierwszej kolejności aplikacja generuje wszystkie standardowe wiersze dla tabeli, całkowicie pomijając kolumny LLM. Działa to natychmiastowo i tworzy gotowe obiekty, które mogą posłużyć jako kontekst dla zapytań AI.
2. Zebrane konteksty są następnie przekazywane w paczkach do oddzielnej funkcji, która zajmuje się wyłącznie komunikacją z modelem.

Kluczem do przyspieszenia procesu okazało się grupowanie zapytań w mniejsze pakiety. Zamiast wysyłać pojedynczy wiersz, program opiera się na parametrze `SUB_BATCH_SIZE` (domyślnie 15) i wysyła do modelu kilkanaście kontekstów w jednym dużym prompcie. W odpowiedzi model zwraca tekstową strukturę JSON zawierającą od razu cały pakiet wyników, co skraca narzut czasu potrzebny na wielokrotne inicjalizowanie zapytań przez silnik AI.

Dodatkowo zastosowano mechanizm `asyncio.Semaphore`, który zezwala serwerowi Ollama na przetwarzanie kilku takich paczek jednocześnie (domyślnie 8 zapytań w tle). Jeśli wystąpi błąd formatowania dla danej paczki (np. model pominie znak w odpowiedzi JSON), uszkodzony fragment jest wyłapywany i przerabiany bezpiecznym, pojedynczym zapytaniem, aby nie psuć całego postępu generowania.

6.3.2 Dobór parametrów optymalizacji

W celu dobrania optymalnych wielkości paczek przeprowadzono testy wydajnościowe. Jako stację wykorzystano laptopa z kartą graficzną NVIDIA RTX 5060, korzystając z modelu Llama 3 uruchomionego lokalnie w środowisku Ollama dla próbki 100 wierszy AI.

Numer próby	Czas [s]	Czas na 1 wiersz [s]	Zmiana w konfiguracji
1	129,52	1,30	Generowanie pojedyncze (sekwencyjne)
2	218,88	2,19	Zbyt wysoka współbieżność zapytań
3	63,43	0,63	Implementacja pakietowania (batches)
4	124,41	1,24	Testowanie innych rozmiarów limitów
5	52,24	0,52	Optymalnie dopasowane parametry

Tabela 4. Czasy generowania 100 wierszy AI na laptopie RTX 5060

Zgodnie z Tabelą 4, przy najlepiej dobranych parametrach udało się skrócić czas ze 130 sekund do około 52. Oznacza to blisko 2,5-krotne przyspieszenie bez negatywnego wpływu na złożoność całego programu. Zaobserwowano również, że zbyt wysokie ustawienie równoległych żądań (próba 2) doprowadziło do przeciążenia API i drastycznego wydłużenia pracy.

6.3.3 Porównanie wydajności Faker i LLM

Kolejne pomiary wykonano na tym samym sprzęcie. Wykorzystano czterotabelowy schemat testowy `ecommerce_llm_schema`, w tym tabelę recenzji z polem `review_text` generowanym przez LLM.

Typ danych	Liczba wierszy	Czas (Faker)	Czas (LLM)
Proste (imię, adres)	1 000	0,8s	–
Proste	10 000	5,2s	–
Złożone (z kontekstem AI)	100	–	52,24s (po optymalizacji)
Złożone (z kontekstem AI)	10 000	–	8 038s ($\approx$ 2h 14min)

Tabela 5. Zestawienie czasów generowania danych strukturalnych i kontekstowych

Jak widać w Tabeli 5, generowanie danych przy użyciu algorytmów pseudolosowych (Faker) jest niezwykle szybkie i skalowalne liniowo. Wąskim gardłem systemu pozostaje generowanie treści przez LLM, co potwierdza słuszność decyzji o zastosowaniu przetwarzania wsadowego z kontrolą współbieżności.

6.3.4 Alternatywny silnik — vLLM

Początkowe prace nad aplikacją oraz testy lokalne opierały się na serwerze Ollama [21] — narzędziu zaprojektowanym z myślą o prostocie obsługi i łatwym uruchamianiu modeli LLM na komputerach osobistych. Ollama wyróżnia się intuicyjnym interfejsem i prostym procesem instalacji, co doskonale sprawdza się przy deweloperskich eksperymentach. Jednakże w zastosowaniach chmurowych, wymagających dużej przepustowości i serwowania wielu równoczesnych zapytań, jego architektura nie oferuje optymalnej wydajności.

W celu maksymalizacji prędkości generowania w środowisku chmurowym przeprowadzono testy z wykorzystaniem vLLM [11] — biblioteki serwującej modele LLM, opracowanej na Uniwersytecie Kalifornijskim w Berkeley. vLLM różni się od Ollama podejściem architektonicznym: zamiast interfejsu zorientowanego na użytkownika końcowego, oferuje zaawansowany silnik zoptymalizowany pod kątem produkcyjnej wydajności i maksymalnego wykorzystania sprzętu.

Kluczowym mechanizmem vLLM jest *PagedAttention* — algorytm zarządzania pamięcią GPU inspirowany mechanizmami stronicowania pamięci z systemów operacyjnych. Standardowe podejście do obsługi mechanizmu uwagi [31] w modelach LLM wymaga alokacji ciągłego bloku pamięci dla każdego zapytania, co prowadzi do fragmentacji i marnotrawstwa zasobów GPU. *PagedAttention* dzieli pamięć kontekstu KV-cache na małe, nieciągłe bloki, które mogą być dynamicznie przydzielane i zwalniane. Drugą istotną zaletą jest mechanizm *continuous batching*, który dynamicznie łączy napływające zapytania w czasie rzeczywistym, bez konieczności czekania na skompletowanie całej paczki.

Konfiguracja vLLM na platformie Kaggle

Wdrożenie vLLM w środowisku Kaggle wymagało rozwiązania dodatkowych problemów środowiskowych. Domyślna wersja vLLM (0.17) korzysta z biblioteki FlashInfer, która wymaga kompilacji kerneli CUDA w czasie wykonywania programu. System plików Kaggle, będący w dużej części środowiskiem tylko do odczytu, uniemożliwiał tę kompilację, powodując błędy budowania oprogramowania.

Rozwiązaniem okazało się zastosowanie starszej wersji vLLM (0.6.6), która zamiast FlashInfer wykorzystuje bibliotekę xFormers [14]. xFormers dostarcza prekompilowane, wydajne implementacje operacji uwagi, co eliminuje konieczność kompilacji kerneli CUDA w czasie uruchamiania serwera.

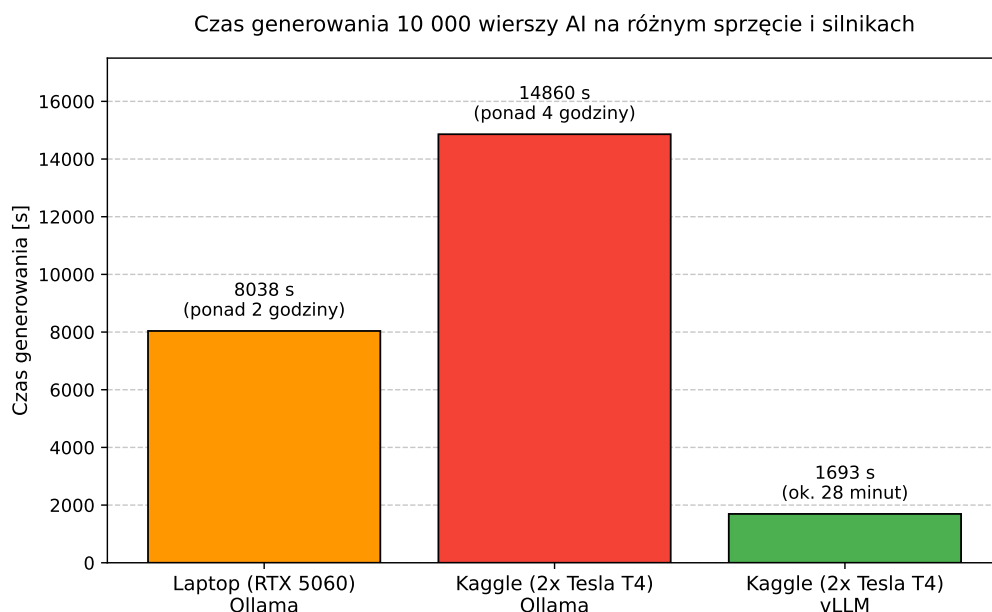
Model Llama 3 8B-Instruct uruchomiono z podziałem tensorów na dwie karty Tesla T4 (`tensor-parallel-size=2`), co pozwoliło na wykorzystanie łącznie 32 GB pamięci VRAM. Aplikacja, dzięki oparciu komunikacji na standardzie OpenAI API, wymagała jedynie zmiany adresu URL serwera modelu oraz wyłączenia sztucznego podziału na paczki — dotychczasowy interfejs i logika przetwarzania po stronie backendu pozostały niezmienione, delegując pełne zarządzanie ruchem do silnika vLLM.

6.3.5 Skalowalność dla dużych zbiorów danych

W celu ostatecznego porównania wydajności wykonano próbę obciążeniową polegającą na wygenerowaniu bazy danych zawierającej 10 000 wierszy przetworzonych przez LLM. Testy przeprowadzono na dwóch platformach sprzętowych oraz z dwoma silnikami inferencji (Ollama i vLLM).

Środowisko testowe	Silnik	Wierszy AI	Czas [s]	Czas/wiersz [s]
Laptop (RTX 5060)	Ollama	100	52,24	0,52
Laptop (RTX 5060)	Ollama	10 000	8 038,12	0,80
Kaggle (2x Tesla T4)	Ollama	100	178,02	1,78
Kaggle (2x Tesla T4)	Ollama	10 000	14 860,23	1,49
Kaggle (2x Tesla T4)	vLLM	100	18,30	0,18
Kaggle (2x Tesla T4)	vLLM	10 000	1 693,50	0,17

Tabela 6. Porównanie skalowalności w zależności od sprzętu i silnika



Rysunek 12. Wykres porównujący czas generowania 10 000 wierszy AI na różnym sprzęcie

Jak przedstawia Rysunek 12, wygenerowanie 10 000 wierszy na laptopie z serwerem Ollama zajęło ponad 8 038 sekund (nieco ponad 2 godziny). Długotrwałe obciążenie karty graficznej wiązało się ze zjawiskiem tzw. *thermal throttling* (zabezpieczenie przed przegrzaniem obniżające taktowanie procesora), przez co średni czas utworzenia jednego wiersza wzrósł z 0,52 do 0,80 sekundy. Mimo to aplikacja zniosła ten test poprawnie i zapisała całą tabelę w pliku bazy SQLite.

Dla porównania, ten sam schemat z serwerem Ollama uruchomiono w darmowym środowisku chmurowym Kaggle, dysponującym dwoma kartami Tesla T4 (łącznie 32 GB pamięci VRAM). Pełna próba 10 000 wierszy na Kaggle zajęła aż 14 860 sekund (ponad 4 godziny). Różnica wydajności względem laptopa wynika ze specyfikacji sprzętowych — karty Tesla T4 oparte są na architekturze Turing [19] z 2018 roku, która nie posiada ulepszonych jednostek obliczeniowych obecnych w nowoczesnych kartach z serii RTX (architektura Ada Lovelace [20]).

Zastąpienie serwera Ollama przez vLLM na tej samej platformie Kaggle przyniosło drastyczną poprawę wydajności. Generowanie 100 wierszy AI skróciło się ze 178 do zaledwie 18,3 sekundy (przyspieszenie blisko 10-krotne), a pełna próba 10 000 wierszy ze 14 860 do 1 693 sekund (około 28 minut zamiast ponad 4 godzin) — przyspieszenie blisko 9-krotne. Średni czas na wiersz z vLLM (0,17 s) okazał się ponad 4-krotnie lepszy niż najlepsza lokalna konfiguracja Ollama na laptopie RTX 5060 (0,52 s). Wynik ten potwierdza, że mechanizmy PagedAttention i *continuous batching* zaimplementowane w vLLM pozwalają na znacznie efektywniejsze wykorzystanie chmurowych zasobów GPU niż tradycyjne podejście serwera Ollama.

6.4 Podsumowanie i wnioski

Celem niniejszej pracy magisterskiej było zaprojektowanie i zaimplementowanie nowoczesnego narzędzia do generowania syntetycznych danych relacyjnych, wspieranego przez modele sztucznej inteligencji. Cel ten został osiągnięty poprzez stworzenie kompletnej aplikacji webowej, która integruje szybkość tradycyjnych generatorów z kreatywnością modeli LLM.

6.4.1 Osiągnięcia

W ramach prac zrealizowano wszystkie założone wymagania funkcjonalne i нефункционалne:

- **Hybrydowy silnik generujący:** Stworzono elastyczny mechanizm łączący bibliotekę Faker, szablony Jinja2 oraz modele LLM (OpenAI/Ollama). Pozwala to na generowanie danych o dowolnym stopniu skomplikowania — od prostych typów liczbowych po złożone opisy tekstowe.
- **Obsługa relacji:** Zaimplementowano algorytm automatycznego rozwiązywania zależności między tabelami, co gwarantuje spójność referencyjną generowanych zbiorów danych.
- **Wydajność:** Dzięki zastosowaniu architektury asynchronicznej (Celery + Redis) oraz optymalizacji wsadowego generowania przez LLM, system jest w stanie obsłużyć generowanie dużych zbiorów danych bez blokowania interfejsu użytkownika.
- **Eksport i integracja z bazami danych:** Zaimplementowano bezpośredni zrzut danych do PostgreSQL, MySQL, MSSQL oraz Oracle, a także eksport do plików JSON i CSV.
- **Konteneryzacja:** Całość rozwiązania została zamknięta w kontenerach Docker, co drastycznie upraszcza proces wdrożenia i konfiguracji w dowolnym środowisku.

Kod źródłowy aplikacji udostępniono w repozytorium GitHub (Załącznik 1).

6.4.2 Dalszy rozwój aplikacji

Zrealizowany projekt stanowi solidną podstawę do dalszego rozwoju. Zidentyfikowano kilka obszarów, w których system mógłby zostać rozbudowany:

1. **Wsparcie dla baz NoSQL:** Rozszerzenie mechanizmu eksportu o silniki dokumentowe (np. MongoDB) zwiększyłoby uniwersalność narzędzia w scenariuszach pozarelacyjnych.
2. **Generowanie na podstawie istniejącej bazy:** Ciekawą funkcjonalnością byłaby inżynieria wsteczna — podłączenie się do istniejącej bazy danych, automatyczne odczytanie jej schematu i wygenerowanie danych testowych „podobnych” do danych produkcyjnych (z zachowaniem anonimizacji).
3. **Inteligentne wykrywanie typów:** Wykorzystanie LLM do automatycznej sugestii typów danych na podstawie samej nazwy kolumny (np. wpisując nazwę „pesel”, system sam proponuje odpowiedni generator regex).

Podsumowując, stworzone narzędzie wypełnia lukę na rynku generatorów danych testowych, oferując unikalną możliwość generowania treści semantycznie poprawnych i kontekstowych, co jest kluczowe w dobie nowoczesnych aplikacji opartych na danych.

Bibliografia

- [1] Django Software Foundation, *Django Documentation*, Dostęp: 2026-02-17, 2024. adr.: <https://docs.djangoproject.com/>.
- [2] Docker Inc., *Docker Documentation*, Building, shipping, and running distributed applications, 2024. adr.: <https://docs.docker.com/>.
- [3] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*, 2016. adr.: <https://data.europa.eu/eli/reg/2016/679/oj>.
- [4] Faker-js, *Faker*, ver. 9.3.0, Dostęp: 2026-01-10, 2024. adr.: <https://github.com/faker-js/faker>.
- [5] Fielding, R. T., „Architectural Styles and the Design of Network-based Software Architectures”, University of California, Irvine, Ph.D. Dissertation, 2000. adr.: https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf.
- [6] Google, *Angular - The modern web developer's platform*, Dostęp: 2026-02-17, 2024. adr.: <https://angular.dev/>.
- [7] Gretel Labs, Inc. „Gretel: The Synthetic Data Platform for Privacy-Preserving AI”. Dostęp: 2026-01-10. (2024), adr.: <https://gretel.ai>.
- [8] Haerder, T. i Reuter, A., *Principles of Transaction-Oriented Database Recovery*. ACM Computing Surveys, 1983, t. 15, s. 287–317.
- [9] Jones, M., Bradley, J. i Sakimura, N., „JSON Web Token (JWT)”, Internet Engineering Task Force (IETF), RFC 7519, 2015. adr.: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [10] Kahn, A. B., „Topological sorting of large networks”, *Communications of the ACM*, t. 5, nr. 11, s. 558–562, 1962. DOI: 10.1145/368996.369025.
- [11] Kwon, W. i in., „Efficient Memory Management for Large Language Model Serving with PagedAttention”, w *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, ACM, 2023. DOI: 10.1145/3600006.3613165.
- [12] MDN Contributors, *JavaScript modules - JavaScript | MDN*, Dostęp: 2026-02-17, 2025. adr.: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>.

- [13] Meta Platforms, Inc., *React - A JavaScript library for building user interfaces*, The library for web and native user interfaces, 2024. adr.: <https://react.dev/>.
- [14] Meta Research, *xFormers: A modular and hackable Transformer modelling library*, Dostęp: 2026-03-18, 2022. adr.: <https://github.com/facebookresearch/xformers>.
- [15] MOSTLY AI, „MOSTLY AI: The Synthetic Data Platform”. Dostęp: 2026-01-10. (2024), adr.: <https://mostly.ai>.
- [16] Mozilla Developer Network, *SPA (Single-page application) - MDN Web Docs Glossary*, [Dostęp: 19-March-2026], 2024. adr.: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
- [17] Nadăș, M., Dioșan, L. i Tomescu, A., „Synthetic Data Generation Using Large Language Models: Advances in Text and Code”, *IEEE Access*, t. 13, s. 134 615–134 633, 2025, ISSN: 2169-3536. DOI: 10.1109/access.2025.3589503. adr.: <http://dx.doi.org/10.1109/ACCESS.2025.3589503>.
- [18] Narayanan, A. i Shmatikov, V., „Robust De-anonymization of Large Sparse Datasets”, w *2008 IEEE Symposium on Security and Privacy (sp 2008)*, IEEE, 2008, s. 111–125. DOI: 10.1109/SP.2008.33.
- [19] NVIDIA Corporation, „NVIDIA Turing Architecture Whitepaper”, NVIDIA, spraw. tech., 2018. adr.: <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>.
- [20] NVIDIA Corporation, „NVIDIA Ada Lovelace Architecture Whitepaper”, NVIDIA, spraw. tech., 2022. adr.: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>.
- [21] Ollama, *Ollama - Get up and running with Llama 2 and other large language models*, Local LLM runner, 2024. adr.: <https://ollama.ai/>.
- [22] Ramírez, S., *FastAPI Documentation*, High performance, easy to learn, fast to code, ready for production, 2024. adr.: <https://fastapi.tiangolo.com/>.
- [23] Redis Ltd., *Redis Documentation*, The in-memory data structure store, 2024. adr.: <https://redis.io/docs/>.
- [24] Rollup team, *Rollup - The JavaScript module bundler*, Dostęp: 2026-02-17, 2024. adr.: <https://rollupjs.org/>.
- [25] Ronacher, A. i contributors, *Flask Documentation*, Dostęp: 2026-02-17, 2024. adr.: <https://flask.palletsprojects.com/>.
- [26] Semgrep, Inc., *Semgrep - Static Code Analysis Tool*, Dostęp: 2026-02-17, 2024. adr.: <https://semgrep.dev/>.
- [27] Silberschatz, A., Korth, H. F. i Sudarshan, S., *Database System Concepts*, 7th. McGraw-Hill Education, 2019, ISBN: 9780078022159.

- [28] Silberschatz, A., Korth, H. F. i Sudarshan, S., *Database System Concepts*, 7th. McGraw-Hill Education, 2019, ISBN: 9780078022159.
- [29] Solem, A. i contributors, *Celery - Distributed Task Queue*, Distributed Task Queue, 2024. adr.: <https://docs.celeryq.dev/>.
- [30] The PostgreSQL Global Development Group, *PostgreSQL Documentation*, The World's Most Advanced Open Source Relational Database, 2023. adr.: <https://www.postgresql.org/docs/>.
- [31] Vaswani, A. i in., „Attention is All you Need”, w *Advances in Neural Information Processing Systems*, t. 30, Curran Associates, Inc., 2017.
- [32] Wikipedia contributors, *Air gap (networking)* — *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Air_gap_\(networking\)&oldid=1331234099](https://en.wikipedia.org/w/index.php?title=Air_gap_(networking)&oldid=1331234099), [Online; accessed 10-January-2026], 2026.
- [33] Wikipedia contributors, *Boilerplate code* — *Wikipedia, The Free Encyclopedia*, [Online; accessed 4-March-2026], 2026. adr.: https://en.wikipedia.org/w/index.php?title=Boilerplate_code&oldid=1335261906.
- [34] Wikipedia contributors, *Happy path* — *Wikipedia, The Free Encyclopedia*, [Online; accessed 15-February-2026], 2026. adr.: https://en.wikipedia.org/w/index.php?title=Happy_path&oldid=1336934215.
- [35] You, E. i contributors, *Vite - Next Generation Frontend Tooling*, Dostęp: 2026-02-17, 2024. adr.: <https://vitejs.dev/>.
- [36] You, E. i contributors, *Vue.js - The Progressive JavaScript Framework*, Dostęp: 2026-02-17, 2024. adr.: <https://vuejs.org/>.

Wykaz skrótów i symboli

API Application Programming Interface 13, 18, 19, 22, 23, 24, 25, 26, 34, 35, 39, 40, 41

CI/CD Continuous Integration / Continuous Deployment 9

CORS Cross-Origin Resource Sharing 42

CRUD Create, Read, Update, Delete 39

CSS Cascading Style Sheets 25

CSV Comma-Separated Values 10, 18, 27, 34, 48

DOM Document Object Model 15

E2E End-to-End 7, 39, 41, 63

HMR Hot Module Replacement 15

HTTP Hypertext Transfer Protocol 15, 25, 42

JSON JavaScript Object Notation 9, 10, 11, 15, 18, 27, 34, 36, 39, 44, 48

JWT JSON Web Token 25, 27, 39

LLM Large Language Model 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 24, 27, 29, 35, 36, 37, 39, 41, 42, 43, 45, 46, 48

REST Representational State Transfer 22, 23

SaaS Software as a Service 10

SAST Static Application Security Testing 41

SCA Software Composition Analysis 41

SPA Single Page Application 14, 25

SQL Structured Query Language 9, 10, 11, 15, 18, 27, 32, 34

UI User Interface 22, 24

UUID Universally Unique Identifier 17

UX User Experience 22

Spis rysunków

1	Schemat architektury systemu	23
2	Ekran logowania do systemu	26
3	Formularz konfiguracji globalnej i pasek narzędzi	28
4	Menedżer tabel	28
5	Formularz definiowania nowego pola	29
6	Formularz definiowania nowego pola z wykorzystaniem AI	30
7	Formularz definiowania nowego pola z wykorzystaniem rozkładu prawdopodobieństwa	31
8	Interfejs konfigurowania klucza obcego pośród opcji parametrów w panelu bocznym	32
9	Lista pól dla aktywnej tabeli	33
10	Okno postępu generowania	33
11	Okno eksportu danych do bazy danych	34
12	Wykres porównujący czas generowania 10 000 wierszy AI na różnym sprzęcie	47

Spis tabel

1	Porównanie narzędzi do generowania danych	11
2	Porównanie frameworków webowych Python	14
3	Porównanie technologii frontendowych	14
4	Czasy generowania 100 wierszy AI na laptopie RTX 5060	44
5	Zestawienie czasów generowania danych strukturalnych i kontekstowych	45
6	Porównanie skalowalności w zależności od sprzętu i silnika	46

Spis załączników

1	Kod źródłowy aplikacji w repozytorium GitHub	63
---	--	----

Załącznik 1

Kod źródłowy aplikacji w repozytorium GitHub

Kompletne rozwiązanie programistyczne opisane w niniejszej pracy — wraz z konfiguracją Docker, testami automatycznymi oraz dokumentacją uruchomieniową — udostępnione jest publicznie w repozytorium:

<https://github.com/jakub-prokopiuk/datasynth-ai>

Repozytorium zawiera m.in.:

- kod warstwy backendowej (FastAPI, Celery) oraz frontendowej (React),
- pliki `docker-compose.yml` i obrazy Docker niezbędne do uruchomienia systemu,
- testy backendu (pytest) oraz testy E2E (Playwright),
- przykładowe schematy danych wykorzystywane podczas testów wydajnościowych.

Aby uruchomić aplikację lokalnie, wystarczy sklonować repozytorium i wykonać polecenie `docker compose up --build`. Szczegółowy opis konfiguracji znajduje się w pliku `README.md` repozytorium.